

Submissão — Bruno Milford — Challenge 001

Sobre mim

- **Nome:** Bruno Milford
 - **LinkedIn:** [INSERIR LINK DO LINKEDIN]
 - **Challenge escolhido:** Challenge 001 — Diagnóstico de Churn da Raven-Stack
-

Executive Summary

Desenvolvi uma solução analítica para investigar o aumento do churn da Raven-Stack, cruzando dados de contas, assinaturas, uso de funcionalidades, suporte e eventos de cancelamento. Além da análise, construí um protótipo de dashboard interativo em Python, utilizando um banco SQLite consolidado, para transformar os dados em uma ferramenta de acompanhamento operacional. A principal conclusão é que o crescimento agregado de uso não deve ser interpretado isoladamente: ele pode esconder grupos de clientes com queda de engajamento, aumento de erros, baixa adoção de funcionalidades e sinais de fricção no suporte. A recomendação central é implantar um processo contínuo de identificação de risco, priorizando contas pela combinação entre probabilidade de churn e impacto financeiro.

Solução

A entrega foi estruturada em três componentes complementares:

1. **Base de dados consolidada**, reunindo os cinco arquivos do desafio em um único banco SQLite.
2. **Dashboard interativo**, desenvolvido em Python, para exploração dos indicadores e acompanhamento das contas.
3. **Diagnóstico executivo**, conectando os sinais de produto, suporte, assinatura e receita para orientar as decisões do CEO, do time de Customer Success e do time de Produto.

Abordagem

Comecei pelo entendimento da estrutura e do nível de detalhe de cada conjunto de dados. O principal cuidado foi evitar cruzamentos que multiplicassem registros, pois assinaturas, uso diário, tickets e eventos de churn possuem granularidades diferentes.

A análise foi organizada a partir de uma visão consolidada por `account_id`, utilizando `subscription_id` como chave intermediária para os dados de assi-

natura e uso. Antes da análise, foram avaliados estrutura das colunas, tipos de dados, datas, duplicidades, valores ausentes e cobertura dos relacionamentos.

Em seguida, o problema foi dividido em cinco frentes:

1. **Churn de contas:** quantas contas foram perdidas e em quais segmentos.
2. **Churn financeiro:** quanto de MRR e ARR foi perdido.
3. **Comportamento de uso:** frequência, duração, amplitude de funcionalidades utilizadas, erros e evolução do uso.
4. **Experiência de suporte:** volume de tickets, escalonamentos, tempo de resposta, tempo de resolução e satisfação.
5. **Risco futuro:** identificação de contas ativas que apresentam sinais semelhantes aos observados antes do churn.

Também tratei separadamente a aparente contradição apresentada pelo CEO:

- O time de Produto afirma que o uso cresceu.
- O time de Customer Success afirma que a satisfação está adequada.
- Mesmo assim, o churn aumentou.

A hipótese investigada foi que indicadores agregados estavam escondendo comportamentos diferentes entre segmentos e contas. Por isso, a análise não ficou limitada às médias gerais da empresa.

Resultados / Findings

1. O crescimento agregado de uso não representa todos os clientes O aumento total de utilização da plataforma pode coexistir com deterioração em grupos específicos. Uma parcela de clientes pode estar ampliando fortemente o uso enquanto outra apresenta queda de frequência, concentração em poucas funcionalidades, aumento de erros ou abandono progressivo.

Esse ponto muda a interpretação da afirmação do time de Produto: o crescimento geral pode ser verdadeiro, mas não é suficiente para explicar retenção. Para acompanhar churn, é mais relevante medir a trajetória de uso por conta e por segmento do que observar apenas o volume total da plataforma.

Evidências a inserir:

- Uso total no período: [INSERIR MÉTRICA]
- Variação média de uso entre contas que permaneceram: [INSERIR MÉTRICA]
- Variação média de uso antes do churn: [INSERIR MÉTRICA]
- Screenshot/gráfico: [INSERIR LINK OU CAMINHO]

2. Satisfação média de suporte pode esconder casos críticos Uma média de satisfação considerada aceitável não elimina a existência de contas com tickets recorrentes, escalonamentos, demora de resolução ou problemas técnicos repetidos. Além disso, os clientes que não abrem chamados podem não

estar satisfeitos; eles podem simplesmente reduzir o uso e cancelar sem registrar formalmente a insatisfação.

A conclusão é que o indicador de satisfação deve ser analisado junto com comportamento de uso, recorrência de problemas, tempo de resolução e impacto financeiro da conta.

Evidências a inserir:

- Satisfação média geral: [INSERIR MÉTRICA]
- Satisfação média das contas que churnaram: [INSERIR MÉTRICA]
- Tempo médio de resolução dos churners versus não churners: [INSERIR MÉTRICA]
- Percentual de churners com tickets escalados: [INSERIR MÉTRICA]

3. A causa raiz é formada pela combinação de sinais Os dados sugerem que o churn não deve ser explicado por uma única variável. O risco cresce quando diferentes sinais aparecem simultaneamente, como:

- queda recente de uso;
- baixa adoção de funcionalidades;
- concentração da utilização em poucas features;
- aumento de erros;
- tickets recorrentes ou escalados;
- piora no tempo de resposta ou resolução;
- downgrade recente;
- contrato mensal, com menor barreira para cancelamento;
- baixa ativação após trial;
- valor financeiro relevante exposto ao risco.

O principal insight é que Produto, CS e Receita estavam observando partes diferentes do mesmo problema. O churn aparece na interseção dessas dimensões.

4. Nem todo churn possui o mesmo impacto Além da quantidade de contas canceladas, a solução considera o valor financeiro perdido. Uma conta de baixo MRR e uma conta estratégica de alto MRR não devem receber o mesmo tratamento.

Por isso, o dashboard separa:

- **probabilidade de churn**, baseada nos sinais comportamentais e operacionais;
- **impacto financeiro**, baseado no MRR e ARR da conta.

Essa separação evita priorizar uma conta apenas porque ela possui maior receita e também evita concentrar esforço em muitas contas pequenas sem avaliar o retorno potencial da ação.

Evidências a inserir:

- MRR total perdido: [INSERIR MÉTRICA]
- ARR total perdido: [INSERIR MÉTRICA]
- Receita concentrada nas contas de maior risco: [INSERIR MÉTRICA]
- Lista das contas prioritárias: [INSERIR LINK OU SCREENSHOT]

5. Segmentos e contas em risco precisam de acompanhamento operacional A análise segmenta o risco por plano, indústria, país, canal de aquisição, trial, frequência de cobrança, faixa de MRR/ARR, tempo de relacionamento e comportamento de uso e suporte.

As contas ativas foram classificadas em quatro níveis:

- **Crítico**
- **Alto**
- **Médio**
- **Baixo**

Cada classificação é acompanhada pelos sinais que justificam o risco, permitindo que o time de CS entenda por que uma conta foi priorizada e qual ação deve ser executada.

6. Protótipo construído Foi desenvolvido um dashboard interativo em Python, com HTML e CSS integrados à aplicação e leitura dos dados a partir de um banco SQLite.

O protótipo foi pensado para permitir:

- visão executiva dos indicadores de churn;
- filtros por segmento;
- acompanhamento de MRR e ARR em risco;
- análise de uso e adoção de funcionalidades;
- análise de suporte;
- identificação de contas críticas;
- visualização dos principais fatores de risco;
- apoio à priorização das ações de Customer Success e Produto.

Links e evidências:

- Repositório: [INSERIR LINK DO GITHUB]
- Dashboard: [INSERIR LINK, GIF OU SCREENSHOT]
- Relatório executivo: [INSERIR LINK OU CAMINHO]
- Documentação de execução: [INSERIR LINK OU CAMINHO]

Recomendações

Prioridade 1 — Criar uma rotina operacional de prevenção de churn Implantar uma lista semanal de contas em risco, combinando probabilidade de churn e receita exposta.

- **Público-alvo:** contas classificadas como críticas ou de alto risco.

- **Responsável:** Customer Success.
- **Gatilhos:** queda de uso, erros, tickets recorrentes, escalonamento, downgrade e baixa adoção.
- **Ação:** contato proativo com diagnóstico específico da conta.
- **Métrica:** percentual de contas recuperadas, MRR preservado e redução do churn no grupo tratado.

Prioridade 2 — Monitorar tendência de uso, não apenas volume total

O time de Produto deve acompanhar variações por conta, plano e segmento, especialmente nas janelas de 7, 30 e 90 dias.

- **Responsável:** Produto e Dados.
- **Ação:** criar alertas de queda de uso, abandono de funcionalidades e aumento da taxa de erros.
- **Métrica:** recuperação de uso, expansão da adoção e redução do churn entre contas alertadas.

Prioridade 3 — Integrar suporte ao processo de retenção Tickets escalados, demora de resolução e recorrência de problemas devem alimentar automaticamente o risco da conta.

- **Responsável:** Suporte e Customer Success.
- **Ação:** encaminhar contas com problemas recorrentes para revisão conjunta.
- **Métrica:** redução de reincidência, tempo de resolução e churn após incidentes.

Prioridade 4 — Criar jornadas específicas por segmento Os segmentos não devem receber a mesma estratégia de retenção.

Exemplos:

- clientes mensais: incentivo à migração para contrato anual após comprovação de valor;
- clientes oriundos de trial: jornada de ativação e acompanhamento nos primeiros meses;
- contas com baixa amplitude de uso: treinamento focado nas funcionalidades relevantes;
- contas com alto MRR: plano de sucesso e acompanhamento executivo;
- contas com erros recorrentes: atuação técnica prioritária.

Prioridade 5 — Validar o score antes de automatizar decisões O score deve começar como ferramenta de apoio, com regras explicáveis e validação prática pelo time de CS. Após acumular histórico suficiente, pode ser evoluído para um modelo preditivo com divisão temporal, calibração e acompanhamento de performance.

Limitações

- O conjunto de dados representa um cenário analítico e não contém todo o contexto qualitativo das decisões de cancelamento.
- Alguns campos relacionados ao churn, como motivo informado, feedback e refund, são posteriores ao cancelamento e não devem ser usados como variáveis preditivas.
- Correlações observadas não comprovam causalidade. As recomendações devem ser testadas por meio de ações controladas e acompanhamento de resultados.
- A qualidade do score depende da data de corte e da construção de janelas temporais comparáveis entre churners e não churners.
- O protótipo utiliza SQLite e foi desenvolvido para demonstração. Em ambiente produtivo, seria necessário avaliar autenticação, controle de acesso, segurança, observabilidade, escalabilidade e atualização automatizada dos dados.
- Os números finais, screenshots e links da entrega devem ser inseridos nos campos indicados antes do envio.

Process Log — Como usei IA

Este bloco é obrigatório. Sem ele, a submissão é desclassificada.

Ferramentas usadas

Ferramenta	Para que usou
ChatGPT	Estruturação inicial do problema, discussão de hipóteses, definição das análises, revisão da lógica de churn e preparação do relatório executivo.
OpenAI Codex	Leitura e manipulação dos arquivos, criação dos scripts de processamento, consolidação dos CSVs em SQLite, desenvolvimento do dashboard e ajustes de layout.
Python	Tratamento dos dados, cruzamento das cinco tabelas, cálculos analíticos, construção dos indicadores e execução do protótipo.
SQLite	Armazenamento consolidado dos dados utilizados pelo dashboard.
Git/GitHub	Versionamento do código, registro das alterações e organização da entrega.

Workflow

1. Li o enunciado do challenge e transformei as perguntas do CEO em hipóteses analíticas verificáveis.
2. Revisei a estrutura das cinco bases e identifiquei as chaves `account_id` e `subscription_id`.
3. Validei tipos de dados, datas, duplicidades, nulos e relacionamentos antes de realizar os cruzamentos.
4. Defini uma visão analítica por conta, agregando previamente assinatura, uso e suporte para evitar multiplicação de registros.
5. Analisei churn de contas e churn de receita, separando quantidade de cancelamentos de MRR e ARR perdidos.
6. Comparei uso, erros, adoção de funcionalidades e suporte entre contas que churnaram e contas que permaneceram.
7. Investiguei a contradição entre crescimento de uso, satisfação de suporte e aumento do churn.
8. Estruturei um score de risco explicável e uma classificação operacional das contas.
9. Transformei os cinco arquivos CSV em um único banco SQLite.
10. Utilizei o Codex para construir o dashboard em Python e gerar a interface HTML/CSS.
11. Testei os cálculos, filtros, páginas e consistência das informações apresentadas.
12. Revisei o layout para tornar a solução mais profissional e adequada para apresentação executiva.
13. Estruturei a documentação de execução e o relatório final da submissão.

Onde a IA errou e como corrigi

Durante o desenvolvimento, a IA ajudou a acelerar tarefas, mas não foi utilizada como fonte final de verdade.

Os principais pontos de correção foram:

- **Risco de multiplicação de linhas nos cruzamentos:** ao juntar diretamente assinaturas, uso e tickets, os totais poderiam ser inflados. Corrigi agregando cada tabela no nível adequado antes do relacionamento final.
- **Interpretação de médias agregadas:** análises iniciais poderiam concluir que o uso crescente representava melhora geral. Corrigi segmentando os dados e comparando a trajetória das contas.
- **Uso de informações posteriores ao churn:** motivo de cancelamento, feedback e refund poderiam gerar vazamento de informação em um modelo. Esses campos foram tratados apenas como evidência descritiva, não como preditores.
- **Classificação de risco baseada somente em receita:** alto MRR representa impacto, mas não necessariamente maior probabilidade de churn. Separei risco comportamental de impacto financeiro.

- **Recomendações genéricas:** transformei sugestões amplas em ações com público-alvo, gatilho, responsável e métrica.
- **Layout inicial simples:** o primeiro protótipo cumpria a função, mas precisava de melhor hierarquia visual. O dashboard foi revisado para uma apresentação mais profissional.

O que eu adicionei que a IA sozinha não faria

O principal valor que adicionei foi o julgamento sobre como transformar uma análise técnica em uma ferramenta de decisão.

Não me limitei a pedir um modelo ou um dashboard. Questionei a narrativa apresentada pelos times e defini que o problema deveria ser analisado de forma integrada. A partir disso:

- tratei a contradição entre CEO, CS e Produto como parte central do diagnóstico;
- priorizei explicabilidade, pois o time precisa saber por que uma conta foi classificada como risco;
- separei probabilidade de churn de impacto financeiro;
- adaptei a solução para uso operacional, e não apenas para apresentação;
- revisei as respostas da IA, os cruzamentos, os cálculos e o layout;
- defini que a recomendação final deveria conectar Produto, Suporte e Customer Success em um único processo de retenção.

A IA acelerou a execução, mas as decisões sobre estrutura, prioridade, validação e comunicação foram conduzidas por mim.

Evidências

- ☐ Screenshots das conversas com ChatGPT
- ☐ Screenshots das interações com o Codex
- ☐ Screen recording do dashboard e do workflow
- ☐ Chat exports
- ☐ Git history
- ☐ Repositório no GitHub
- ☐ Screenshots do dashboard
- ☐ Relatório executivo em PDF
- ☐ Documentação técnica e instruções de execução
- ☐ Outro: **Banco SQLite e arquivos de teste**

Submissão enviada em: 20 de julho de 2026

Evidências do processo com IA

As capturas abaixo registram a análise inicial dos cinco arquivos, a criação do prompt para desenvolvimento do dashboard, a revisão dos problemas de lógica e visualização e a solicitação da documentação técnica.

As imagens foram mantidas na ordem cronológica do processo.



ravenstack_accounts(1).xlsx
Planilha



ravenstack_churn_events(1).xlsx
Planilha



ravenstack_feature_usage(1).xlsx
Planilha



ravenstack_subscriptions(1).xlsx
Planilha



ravenstack_support_tickets(1).xlsx
Planilha

Com a skiil criada, analise esses 5 arquivos

Pensou por 2m 28s >

Concluí o cruzamento das cinco bases: **500 contas**, **5.000 assinaturas**, **25.000 registros de uso**, **2.000 tickets** e **600 eventos de churn**.

Principal conclusão

O churn da RavenStack **não parece ser causado principalmente por baixo uso ou insatisfação com o suporte**.

O padrão mais forte é:

churn precoce + desalinhamento entre expectativa, funcionalidades, preço e orçamento.

As evidências principais:

- **339 de 500 contas, ou 67,8%**, possuem evento efetivo de churn não classificado como reativação.
- Aproximadamente **49,6% dos churners cancelaram em até 90 dias**.
- Cerca de **67,2% cancelaram em até 180 dias**.
- A mediana de relacionamento dos churners foi de **93 dias**, contra **298 dias** para contas ainda ativas.
- O uso médio nos 30 dias anteriores ao churn foi de **21,8 interações**, contra **19,5** nas contas ativas.
- A satisfação média nos tickets foi praticamente igual: **4,08 entre churners** e **3,99 entre contas ativas**.
- Os motivos **features**, **pricing**, **budget** e **com**  **tor** correspondem à maior parte dos motivos registrados.

Isso explica a aparente contradição relatada pelo CEO: o uso pode estar crescendo e o suporte pode ter boa avaliação, enquanto clientes novos continuam cancelando por não perceberem valor suficiente ou por encontrarem problemas de adequação comercial e funcional.

Segmentos mais expostos

- **DevTools:** churn de 78,8%, com 89 cancelamentos em 113 contas.
- **Plano Pro:** churn de 78,7% e aproximadamente **US\$ 346 mil de MRR perdido**.
- **Reino Unido:** churn de 79,3%, embora com uma base menor.
- **Canal "other":** churn de 79,6%, indicando baixa qualidade de aquisição ou classificação inadequada do canal.
- **Cobrança mensal:** churn de 77,1%, contra 73,1% no anual.

O efeito mais relevante, porém, continua sendo o período inicial da jornada, especialmente os primeiros 90 dias.

Problema crítico de dados

Existe uma inconsistência importante entre as tabelas:

- **339 contas** possuem evento efetivo de churn;
- apenas **110 contas** estão marcadas com `churn_flag=true` no cadastro;
- **265 contas** possuem evento de churn, mas não possuem a flag;
- **36 contas** possuem a flag, mas não possuem evento correspondente.

Antes de automatizar dashboards ou criar um modelo preditivo oficial, a RavenStack precisa definir uma única regra de churn.

Contas ativas prioritárias

Entre as contas com maior combinação de risco e impacto financeiro estão:

Conta	Empresa	Score	MRR
A-d4e0d4	Company_403	55	US\$ 23.283
A-9badbd	Company_63	30	US\$ 11.741
A-1f7acb	Company_281	25	US\$ 12.736
A-297a2d	Company_448	55	US\$ 5.373
A-7d514c	Company_119	85	US\$ 3.383

A-1986ca	Company_197	65	US\$ 3.980
A-b48f73	Company_225	60	US\$ 3.920

O score é heurístico e explicável, baseado em sinais como queda de uso, baixa amplitude de features, erros, tickets, cobrança mensal, renovação e downgrade. Ele ainda não deve ser tratado como uma probabilidade estatística calibrada.

Ações recomendadas

A primeira prioridade deve ser criar um **programa de ativação dos primeiros 90 dias**, com marcos de valor nos dias 14, 30 e 60.

Em seguida:

1. Melhorar a qualificação comercial, registrando funcionalidades indispensáveis, orçamento e critérios de sucesso.
2. Bloquear vendas dependentes de funcionalidades inexistentes sem aceite formal do cliente.
3. Criar revisões de ROI antes dos dias 60 e 90.
4. Monitorar semanalmente contas com queda de uso e baixa adoção de features.
5. Corrigir a regra e a sincronização do `churn_flag`.

Arquivos gerados

 [relatorio_diagnostico_churn.md](#)

 [diagnostico_churn_ravenstack.xlsx](#)

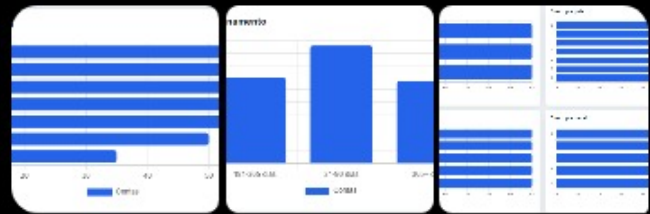
 [process_log.md](#)

     ...

Beleza, agora cria um prompt para o codex, aonde ele vai ser acesso a esses dados para criar um dashboard interativo com html, css e js, passando também o acesso aos arquivos .xlsx.

Abaixo está um prompt completo para enviar ao Codex junto com os cinco arquivos `.xlsx`.





Não gerou um resultado legal, gerou graficos muito complexos e a logica aplicada tem que ser melhor explicada ou refeita, na imagem anexada eu tenho os motivos, os motivos não estão escritos, eles estão por numero o que para um CEO que está analisando vai ficar confuso

Outro exemplo, no Churn por tempo de relacionamento a logica de tempo está errada, como pula de 0-30 dias para 181-365 dias

E quando clico na outras poaginas segue imagem de exemplo, como pode estar tudo maximizado

Preciso de um prompt para o codex, simples e direto, para ele olhar, analisar e ajustar

Mostrar menos ^

Use este prompt no Codex:

Revise o dashboard de churn já criado e corrija os problemas de visualização e lógica mostrados nas imagens. 

Não recrie o projeto do zero. Analise o código atual, identifique a causa dos problemas e faça ajustes para corrigi-los.

Problemas que precisam ser corrigidos:

1. Rótulos dos gráficos

- Não exibir categorias como 0, 1, 2, 3.
- Usar os nomes reais dos dados.
- No gráfico "Principais motivos de churn", mostrar textos como:
 - Pricing
 - Features
 - Budget
 - Competitor
 - Support
 - Other
- Aplicar a mesma correção em plano, país, indústria, canal e demais gráficos.
- Revisar o processamento dos objetos usados pelo Chart.js. Os labels devem vir das categorias reais e o

2. Ordem do tempo de relacionamento

O gráfico "Churn por tempo de relacionamento" está fora de ordem.

Usar exatamente esta ordem:

- 0-30 dias
- 31-90 dias
- 91-180 dias
- 181-365 dias
- Mais de 365 dias

Não ordenar essas faixas alfabeticamente nem pelo valor do gráfico. Criar uma ordem fixa no código.

3. Cálculo correto da taxa de churn

Os gráficos de churn por plano, país, indústria e canal não podem mostrar 100% para todos os segmentos.

Para cada segmento, calcular:

$\text{taxa de churn} = \text{contas com churn no segmento} / \text{total de contas do segmento}$

Exemplo:

Plano Pro:

- 80 contas no total
- 50 contas com churn
- taxa = 62,5%

Não dividir churn por churn e não considerar apenas as contas canceladas no denominador.

Mostrar no tooltip:

- nome do segmento;
- contas com churn;
- total de contas;
- taxa de churn.

4. Definição única de churn

Revise a forma como o dashboard identifica uma conta com churn.

Use uma função central, por exemplo:

`isChurned(accountId)`

Ela deve aplicar uma única regra em todas as páginas e gráficos.



Preferencialmente, considerar como churn uma conta que tenha evento efetivo de churn na tabela de evento.

5. Tamanho dos gráficos

Os gráficos estão ocupando espaço demais e ficando esticados.

Corrigir o layout para:

- altura padrão entre 280px e 340px;
- no máximo dois gráficos por linha no desktop;
- um gráfico por linha em telas menores;
- manter aspect ratio controlado;
- impedir crescimento infinito do canvas;
- evitar gráficos muito largos ou muito altos.

Usar containers como:

```
.chart-card {  
  min-height: 320px;  
  max-height: 380px;  
}  
  
.chart-card canvas {  
  width: 100% !important;  
  height: 280px !important;  
}
```

No Chart.js, usar:

```
responsive: true,  
maintainAspectRatio: false
```

6. Legibilidade executiva

O dashboard será usado por um CEO.

Portanto:

- usar nomes completos;
- evitar códigos internos;
- evitar números sem explicação;
- limitar gráficos a no máximo 8 ou 10 categorias;
- ordenar os gráficos de forma lógica;
- mostrar valores e percentuais nos tooltips;
- apresentar título e breve descrição em cada gráfico;
- usar linguagem simples.

7. Principais motivos de churn

Corrigir especificamente esse gráfico.

Deve mostrar:

- motivo em texto no eixo;
- quantidade de contas;



Deve mostrar:

- motivo em texto no eixo;
- quantidade de contas;
- ordenação da maior para a menor quantidade;
- tooltip com quantidade e percentual sobre o total de churners;
- sem legenda genérica "Contas" se ela não agregar informação.

8. Gráficos por segmento

Para plano, país, indústria e canal:

- ordenar da maior taxa de churn para a menor;
- exibir somente segmentos com dados válidos;
- mostrar o tamanho da amostra;
- não destacar segmentos pequenos sem indicar o total de contas;
- usar percentual de 0% a 100% no eixo;
- manter os nomes das categorias visíveis.

9. Investigar a origem do erro

Antes de alterar, verifique:

- se labels e values estão invertidos;
- se Object.keys e Object.values estão sendo usados no objeto correto;
- se o agrupamento está retornando índices de array em vez dos nomes;
- se o cálculo está sendo feito depois de filtrar apenas churners;
- se existe ordenação alfabética indevida das faixas;
- se o canvas está herdando altura incorreta do container.

10. Validação obrigatória

Depois de corrigir:

- abrir todas as páginas;
- testar os filtros;
- confirmar que nenhum gráfico mostra índices numéricos como categorias;
- confirmar que as faixas de tempo estão na ordem correta;
- confirmar que plano, país, indústria e canal não apresentam 100% para todos;
- verificar se os tooltips mostram numerador, denominador e taxa;
- testar em desktop e tablet;
- garantir que não existam erros no console.

Não fixe os resultados manualmente no HTML.

Os labels, totais e percentuais devem continuar sendo calculados a partir dos arquivos Excel.

Ao finalizar, informe:

- quais arquivos foram alterados;
- qual era a causa de cada erro;
- qual regra de churn ficou definida;
- como a taxa de churn passou a ser calculada;
- como foi corrigida a responsividade dos gráficos.



ontem 15:49

Agora com o projeto já criado, testado, validado, né, revisto, o que que a gente precisa fazer? A gente precisa criar agora a documentação, né? A documentação, o que que ela vai ter, né? Ela vai ter que precisa estar lá, né, tudo, literalmente, como executar o projeto, né, e também o que que esse projeto aborda e faz, tá? Isso, eu preciso que tu crie um prompt pra mim, né, com isso tudo que eu falei, e pra gente mandar pro CodeX pra ele fazer isso.

Segue um prompt completo para o Codex criar a documentação técnica do projeto:

 Editar

Atue como um engenheiro de software sênior e especialista em documentação técnica.

O projeto já foi desenvolvido, testado, revisado e validado. Agora sua tarefa é analisar completamente o repositório e criar uma documentação profissional, clara, detalhada e fiel ao funcionamento real da aplicação.

Objetivo

Criar a documentação oficial do projeto, explicando:

1. Qual problema o projeto resolve;
2. Qual é o objetivo da solução;
3. O que o sistema faz;
4. Como a solução está estruturada;
5. Como instalar, configurar e executar o projeto;
6. Como os dados são processados;
7. Como o banco de dados é criado e utilizado;
8. Como utilizar o dashboard;
9. Quais tecnologias foram utilizadas;
10. Quais são as principais regras, indicadores e análises existentes;
11. Como validar se a solução atende...

Apêndice A - Prompt do dashboard

Prompt completo utilizado para orientar o Codex na construção do dashboard interativo de churn da RavenStack.

Desenvolvimento de Dashboard Interativo de Churn — RavenStack

Atue como um desenvolvedor frontend sênior, especialista em visualização de dados, dashboards executivos e análise de churn em empresas SaaS B2B.

Objetivo

Criar uma aplicação web interativa para apresentar o diagnóstico de churn da RavenStack, utilizando apenas:

- HTML5;
- CSS3;
- JavaScript puro;
- biblioteca SheetJS para leitura dos arquivos `.xlsx`;
- Chart.js para gráficos;
- nenhuma dependência de backend.

O dashboard deve funcionar localmente no navegador e permitir que os dados sejam carregados diretamente dos arquivos Excel disponibilizados no projeto.

Arquivos de dados

Utilize os seguintes arquivos:

```
/mnt/data/ravenstack_accounts(1).xlsx  
/mnt/data/ravenstack_subscriptions(1).xlsx  
/mnt/data/ravenstack_feature_usage(1).xlsx  
/mnt/data/ravenstack_support_tickets(1).xlsx  
/mnt/data/ravenstack_churn_events(1).xlsx
```

Também existe uma planilha consolidada com resultados da análise:

```
/mnt/data/diagnostico_churn_ravenstack.xlsx
```

Antes de desenvolver os componentes, abra e inspecione todas as abas, colunas, tipos de dados, chaves e formatos desses arquivos.

Não invente colunas ou valores. Toda informação apresentada deve ser calculada a partir dos arquivos.

Relacionamento entre as tabelas

As bases devem ser relacionadas da seguinte forma:

```
accounts.account_id
├─ subscriptions.account_id
├─ support_tickets.account_id
└─ churn_events.account_id

subscriptions.subscription_id
└─ feature_usage.subscription_id
```

A visão executiva principal deve usar `account_id` como entidade central.

Antes de realizar joins, agregue tabelas de alta cardinalidade, como uso de funcionalidades, tickets e assinaturas, para evitar multiplicação indevida de registros.

Estrutura do projeto

Crie a seguinte estrutura:

```
ravenstack-churn-dashboard/
├─ index.html
├─ css/
│   └─ styles.css
├─ js/
│   ├── app.js
│   ├── data-loader.js
│   ├── data-processing.js
│   ├── charts.js
│   ├── filters.js
│   └─ utils.js
├─ data/
│   ├── ravenstack_accounts.xlsx
│   ├── ravenstack_subscriptions.xlsx
│   ├── ravenstack_feature_usage.xlsx
│   ├── ravenstack_support_tickets.xlsx
│   ├── ravenstack_churn_events.xlsx
│   └─ diagnostico_churn_ravenstack.xlsx
└─ README.md
```

Copie os arquivos fornecidos para a pasta `data/`, preservando integralmente os dados.

Não modifique o conteúdo dos arquivos originais.

Carregamento dos dados

Implemente duas formas de carregamento:

1. Carregamento automático

Quando o dashboard for executado por um servidor local, carregar os arquivos da pasta `data/`.

Exemplo:

```
python -m http.server 8000
```

O dashboard deverá ficar disponível em:

```
http://localhost:8000
```

2. Carregamento manual

Adicionar uma área que permita selecionar os arquivos `.xlsx` pelo navegador.

O usuário deve poder carregar os seis arquivos manualmente por meio de campos `input type="file"`.

Apresente claramente:

- arquivos carregados;
- arquivos ausentes;
- quantidade de registros de cada arquivo;
- mensagens de erro;
- status do processamento.

Layout e identidade visual

Criar um dashboard executivo moderno, limpo e responsivo.

Utilizar uma identidade visual inspirada em plataformas SaaS B2B:

- fundo geral claro;
- menu lateral em azul-marinho;
- cards brancos;
- bordas discretas;
- cantos arredondados;
- boa hierarquia tipográfica;
- cores semânticas para risco;
- gráficos legíveis;
- tabelas com cabeçalho fixo;
- layout adaptado para desktop e tablet.

Sugestão de cores:

```
--navy: #0f2747;
--blue: #2563eb;
--light-blue: #dbeafe;
--background: #f4f7fb;
--card: #ffffff;
--text: #172033;
--muted: #64748b;
--success: #16a34a;
--warning: #f59e0b;
--danger: #dc2626;
--critical: #991b1b;
--border: #e2e8f0;
```

Não usar estilos inline. Manter todo o CSS no arquivo `styles.css`.

Navegação

Criar um menu lateral com as seguintes páginas ou seções:

1. Visão Executiva;
2. Análise de Churn;
3. Segmentos;
4. Uso do Produto;
5. Suporte;
6. Receita Perdida;
7. Contas em Risco;
8. Qualidade dos Dados;
9. Metodologia.

A navegação pode ser feita como uma SPA simples, alternando as seções com JavaScript.

Cabeçalho

Adicionar:

- título da página;
- data da última atualização;
- status dos arquivos carregados;
- botão para recarregar os dados;
- botão para limpar filtros;
- botão para exportar os dados filtrados em CSV.

Filtros globais

Criar filtros que sejam aplicados em todas as visualizações:

- período;
- status da conta;
- plano;

- indústria;
- país;
- canal de aquisição;
- frequência de cobrança;
- conta trial ou não;
- faixa de MRR;
- motivo de churn;
- faixa de tempo até o churn;
- classificação de risco.

Adicionar também:

- pesquisa por `account_id`;
- pesquisa por nome da empresa;
- botão “Limpar filtros”;
- indicador da quantidade de contas no resultado filtrado.

Os filtros devem ser encadeados. Ao escolher um país, por exemplo, os demais filtros devem refletir apenas as opções ainda disponíveis.

Página 1 — Visão Executiva

Exibir cards com:

- total de contas;
- contas ativas;
- contas com churn;
- taxa de churn por contas;
- total de assinaturas;
- MRR total;
- MRR perdido;
- ARR perdido;
- receita atualmente em risco;
- tempo mediano até o churn;
- quantidade de contas em risco crítico;
- quantidade de contas em risco alto.

Adicionar variação ou comparação quando os dados permitirem.

Criar os seguintes gráficos:

- churn por mês;
- MRR perdido por mês;
- churn por faixa de tempo de relacionamento;
- distribuição das contas por classificação de risco;
- churn de contas versus churn de receita;
- principais motivos de churn.

Adicionar um bloco de conclusões executivas calculadas dinamicamente.

Exemplos de frases:

O principal período de risco ocorre nos primeiros X dias.

O segmento com maior taxa de churn é X, com Y%.

O plano com maior MRR perdido é X, totalizando Y.

A maior concentração de contas críticas está no segmento X.

Essas frases devem ser calculadas com os dados atuais, e não fixadas no HTML.

Página 2 — Análise de Churn

Apresentar:

- taxa geral de churn;
- churn por mês;
- churn por trimestre;
- churn por tempo de relacionamento;
- churn por plano;
- churn por país;
- churn por indústria;
- churn por canal;
- churn por frequência de cobrança;
- churn de clientes trial versus não trial;
- churn por faixa de MRR.

Comparar churners e contas ativas em:

- volume de uso;
- duração de uso;
- número de funcionalidades utilizadas;
- taxa de erro;
- número de tickets;
- tempo de primeira resposta;
- tempo de resolução;
- satisfação;
- escalonamentos;
- upgrades;
- downgrades.

Sempre mostrar:

- quantidade de contas;
- denominador;
- percentual;
- diferença absoluta;
- diferença relativa, quando aplicável.

Página 3 — Segmentos

Criar uma tabela analítica por segmento contendo:

- dimensão;
- segmento;
- total de contas;
- contas com churn;
- taxa de churn;
- MRR total;
- MRR perdido;
- percentual do MRR perdido;
- tempo mediano até churn;
- uso médio;
- tickets médios;
- satisfação média;
- risco médio.

Permitir alternar a dimensão entre:

- plano;
- indústria;
- país;
- canal de aquisição;
- frequência de cobrança;
- trial;
- faixa de MRR;
- tempo de relacionamento.

Adicionar gráficos de barras horizontais para:

- maior taxa de churn;
- maior MRR perdido;
- maior concentração de contas em risco.

Não destacar segmentos com amostras muito pequenas sem apresentar claramente o tamanho da base.

Página 4 — Uso do Produto

Analisar a tabela de uso por funcionalidades.

Apresentar:

- volume total de uso;
- usuários ou assinaturas ativas;
- uso médio por conta;
- tempo médio de uso;
- quantidade média de funcionalidades adotadas;
- taxa média de erro;
- adoção de funcionalidades beta;

- concentração de uso por funcionalidade.

Criar gráficos para:

- uso por funcionalidade;
- uso ao longo do tempo;
- churners versus não churners;
- taxa de erro por funcionalidade;
- funcionalidades mais utilizadas por contas que churnaram;
- funcionalidades com maior associação ao churn;
- amplitude de adoção por conta.

Quando os dados permitirem, calcular tendências de uso em janelas de:

- 7 dias;
- 30 dias;
- 90 dias.

Evitar usar eventos posteriores à data de churn.

Página 5 — Suporte

Apresentar:

- total de tickets;
- tickets por conta;
- tempo médio de primeira resposta;
- tempo médio de resolução;
- satisfação média;
- percentual de escalonamento;
- volume por categoria;
- volume por prioridade.

Criar comparações entre:

- churners e não churners;
- planos;
- indústrias;
- faixas de MRR;
- contas críticas e demais contas.

Criar uma tabela com as contas que apresentam:

- maior volume de tickets;
- pior satisfação;
- maior tempo de resolução;
- mais escalonamentos;
- combinação de alto MRR e problemas de suporte.

Página 6 — Receita Perdida

Apresentar:

- MRR perdido;
- ARR perdido;
- refunds;
- receita perdida por mês;
- receita perdida por plano;
- receita perdida por indústria;
- receita perdida por país;
- receita perdida por canal;
- receita perdida por motivo de churn.

Criar um gráfico de Pareto mostrando quais contas concentram a maior parcela da receita perdida.

Adicionar uma tabela com:

- account_id;
- empresa;
- plano;
- indústria;
- país;
- canal;
- data do churn;
- motivo;
- MRR;
- ARR;
- refund;
- tempo até churn.

Permitir ordenação e exportação.

Página 7 — Contas em Risco

Utilize inicialmente o score disponível na planilha consolidada, caso exista.

Também implemente um score explicável no JavaScript, utilizando os campos disponíveis.

Separar:

Probabilidade de churn

Considerar sinais como:

- queda recente de uso;
- baixo número de funcionalidades utilizadas;
- aumento de erros;
- tickets recorrentes;
- escalonamentos;

- piora na satisfação;
- maior tempo de resolução;
- downgrade;
- cobrança mensal;
- proximidade da renovação;
- baixa ativação após trial.

Impacto financeiro

Considerar:

- MRR;
- ARR;
- relevância relativa dentro da carteira.

Não classificar uma conta apenas porque possui MRR alto.

Classificar o risco como:

Crítico
Alto
Médio
Baixo

Exibir uma tabela com:

- posição;
- account_id;
- empresa;
- plano;
- indústria;
- país;
- MRR;
- ARR;
- score;
- classificação;
- impacto financeiro;
- tendência de uso;
- número de funcionalidades;
- taxa de erro;
- tickets;
- satisfação;
- escalonamentos;
- downgrade;
- principais sinais de risco;
- ação recomendada.

Adicionar filtros, pesquisa, ordenação e paginação.

Ao clicar em uma conta, abrir um painel lateral ou modal com o detalhamento completo:

- dados cadastrais;
- assinaturas;
- evolução do uso;
- funcionalidades utilizadas;
- tickets;
- histórico de upgrades e downgrades;
- risco;
- motivos que compõem o score;
- recomendação para Customer Success.

Página 8 — Qualidade dos Dados

Apresentar as validações realizadas:

- registros por tabela;
- campos nulos;
- chaves duplicadas;
- datas inválidas;
- valores negativos;
- contas sem assinatura;
- assinaturas sem conta;
- uso sem assinatura;
- tickets sem conta;
- eventos de churn sem conta;
- divergências entre plano da conta e plano da assinatura;
- divergências entre `churn_flag` e eventos de churn.

Destacar a inconsistência encontrada entre cadastro e eventos de churn.

Criar uma tabela específica contendo:

- contas com evento de churn e `churn_flag=false`;
- contas com `churn_flag=true` sem evento correspondente;
- eventos classificados como reativação;
- duplicidades por conta.

Adicionar alertas visuais:

Crítico
Atenção
Informativo

O dashboard não deve esconder inconsistências. Elas devem aparecer de forma clara.

Página 9 — Metodologia

Documentar dentro do próprio dashboard:

- arquivos utilizados;
- chaves de relacionamento;
- definição de churn;
- definição de conta ativa;
- definição de MRR e ARR perdido;
- critérios do score de risco;
- regras das janelas temporais;
- tratamento de dados ausentes;
- prevenção de duplicidade nos joins;
- limitações da análise;
- diferença entre correlação e causalidade.

Gráficos

Utilizar Chart.js.

Criar funções reutilizáveis para:

- gráfico de barras;
- gráfico de linha;
- gráfico de rosca;
- gráfico empilhado;
- gráfico de dispersão;
- gráfico de Pareto.

Os gráficos devem:

- atualizar com os filtros;
- destruir a instância anterior antes de renderizar novamente;
- possuir tooltip;
- mostrar valores formatados;
- exibir estado vazio quando não houver dados;
- ser responsivos;
- não distorcer quando o menu lateral for recolhido.

Formatação

Criar funções utilitárias para:

- moeda;
- percentual;
- números;
- datas;
- dias;
- duração;
- valores ausentes.

Exemplos:

```
formatCurrency(value)
formatPercent(value)
formatNumber(value)
formatDate(value)
formatDays(value)
```

Preservar corretamente valores decimais e datas importadas do Excel.

Tabelas

As tabelas devem possuir:

- ordenação;
- pesquisa;
- paginação;
- cabeçalho fixo;
- destaque por nível de risco;
- exportação CSV;
- estado vazio;
- responsividade.

Evitar inserir milhares de linhas de uma vez no DOM.

Regras analíticas obrigatórias

1. Cruzar efetivamente as cinco tabelas.
2. Não realizar joins diretos entre tabelas fato sem agregação prévia.
3. Usar `account_id` como grão da visão executiva.
4. Usar `subscription_id` para relacionar assinaturas e uso.
5. Não utilizar dados posteriores ao churn em análises preditivas.
6. Não utilizar motivo do churn, feedback ou refund no score de contas ainda ativas.
7. Não chamar correlação de causalidade.
8. Sempre apresentar o tamanho da amostra.
9. Não esconder dados nulos ou inconsistentes.
10. Não fixar números no código.
11. Todos os indicadores devem ser recalculados quando os filtros mudarem.
12. Evitar dupla contagem de MRR ou ARR quando uma conta possuir mais de uma assinatura.
13. Documentar claramente qual regra foi utilizada para consolidar assinaturas.
14. Priorizar contas usando risco e impacto financeiro separadamente.

Segurança e estabilidade

- Validar os arquivos antes do processamento;
- tratar abas inexistentes;
- tratar colunas com nomes diferentes;
- impedir erro total da aplicação por causa de um arquivo inválido;
- exibir mensagens compreensíveis;

- registrar erros relevantes no console;
- não utilizar `eval`;
- não utilizar código ofuscado;
- não enviar os dados para serviços externos.

Todo o processamento deve acontecer localmente no navegador.

README

Criar um `README.md` contendo:

- objetivo do projeto;
- estrutura de pastas;
- bibliotecas usadas;
- como copiar os arquivos;
- como iniciar o servidor local;
- como carregar arquivos manualmente;
- descrição das páginas;
- regras analíticas;
- limitações;
- solução de problemas comuns.

Incluir os comandos:

```
cd ravenstack-churn-dashboard  
python -m http.server 8000
```

Depois acessar:

```
http://localhost:8000
```

Explicar que abrir o `index.html` diretamente pode impedir o carregamento automático dos arquivos por causa das regras de CORS do navegador, mas o carregamento manual deve continuar funcionando.

Critérios de aceite

A entrega será considerada concluída somente quando:

- os seis arquivos Excel forem lidos corretamente;
- as cinco bases originais forem cruzadas;
- os cards forem calculados dinamicamente;
- os filtros atualizarem todos os componentes;
- os gráficos forem interativos;
- a tabela de contas em risco funcionar;
- o detalhamento individual da conta funcionar;
- a aba de qualidade dos dados identificar inconsistências;
- o dashboard funcionar sem backend;
- o layout for responsivo;

- não existirem números fictícios;
- o README estiver completo;
- não houver erros no console durante o fluxo normal.

Validação antes de finalizar

Antes de concluir:

1. Execute o dashboard em servidor local.
2. Verifique se os arquivos são carregados.
3. Compare os totais apresentados com as planilhas.
4. Teste todos os filtros.
5. Teste combinações de filtros sem resultado.
6. Teste a pesquisa por conta.
7. Teste a ordenação e paginação.
8. Teste o modal de detalhes.
9. Teste a exportação CSV.
10. Teste o carregamento manual.
11. Teste a aplicação em resolução desktop e tablet.
12. Revise se existe alguma dupla contagem de receita.
13. Confirme que nenhum dado posterior ao churn foi usado indevidamente.
14. Corrija todos os erros encontrados antes de entregar.

Ao final, apresente:

- resumo dos arquivos criados;
- decisões técnicas;
- regra de consolidação dos dados;
- regra utilizada para definir churn;
- regra do score de risco;
- limitações encontradas;
- instruções para executar o dashboard.

Apêndice B - Prompt de documentação

Prompt completo utilizado para orientar o Codex na criação da documentação técnica e operacional do projeto.

Atue como um engenheiro de software sênior e especialista em documentação técnica.

O projeto já foi desenvolvido, testado, revisado e validado. Agora sua tarefa é analisar completamente o repositório e criar uma documentação profissional, clara, detalhada e fiel ao funcionamento real da aplicação.

Objetivo

Criar a documentação oficial do projeto, explicando:

1. Qual problema o projeto resolve;
2. Qual é o objetivo da solução;
3. O que o sistema faz;
4. Como a solução está estruturada;
5. Como instalar, configurar e executar o projeto;
6. Como os dados são processados;
7. Como o banco de dados é criado e utilizado;
8. Como utilizar o dashboard;
9. Quais tecnologias foram utilizadas;
10. Quais são as principais regras, indicadores e análises existentes;
11. Como solucionar os erros mais comuns;
12. Como um novo desenvolvedor pode entender e executar a solução do zero.

Instrução principal

Antes de escrever qualquer documentação, analise todo o repositório.

Não produza uma documentação genérica ou baseada apenas nos nomes dos arquivos.

Leia e compreenda:

- Código Python;
- Arquivos responsáveis pelo backend e pela API;
- Código HTML gerado ou utilizado;
- CSS;
- JavaScript;
- Templates;
- Scripts de criação e população do banco;
- Banco SQLite;
- Consultas SQL;
- Arquivos CSV utilizados como fonte;
- Arquivos de configuração;
- Dependências;
- Rotas;
- Funções;
- Classes;
- Componentes do dashboard;
- Indicadores;
- Filtros;

- Gráficos;
- Tabelas;
- Modelos de churn;
- Regras de negócio;
- Tratamentos de dados;
- Logs;
- Tratamentos de erro;
- Scripts auxiliares;
- Estrutura de diretórios.

A documentação deve representar exatamente o que existe no projeto.

Não invente comandos, recursos, rotas, indicadores, variáveis de ambiente ou funcionalidades.

Caso encontre algo incompleto, inconsistente ou não utilizado, deixe isso claramente registrado.

Contexto geral da solução

O projeto utiliza dados relacionados a uma empresa SaaS B2B e tem como objetivo apoiar a análise de churn, retenção, receita, comportamento dos clientes, uso do produto, suporte e assinaturas.

A solução pode utilizar arquivos como:

- Contas ou clientes;
- Assinaturas;
- Eventos de churn ou cancelamento;
- Uso de funcionalidades;
- Chamados de suporte.

Os dados são consolidados em um banco SQLite, que serve como fonte para um dashboard interativo desenvolvido em Python.

O dashboard deve permitir a análise executiva e operacional dos dados, incluindo, conforme estiver realmente implementado:

- Churn;
- Retenção;
- Receita perdida;
- Receita em risco;
- Contas em risco;
- Comportamento dos clientes;
- Uso de funcionalidades;
- Volume e características dos chamados de suporte;
- Características das assinaturas;
- Segmentações;
- Indicadores executivos;
- Filtros interativos;
- Gráficos;
- Tabelas;
- Análises preditivas;
- Score ou classificação de risco;

- Recomendações para Customer Success, Produto e Receita.

Utilize esse contexto apenas como orientação. A descrição final deve ser confirmada diretamente no código e nos dados do repositório.

Entregáveis

Crie ou atualize os seguintes documentos.

1. README.md principal

O arquivo `README.md` deve ser a principal porta de entrada do projeto.

Utilize uma estrutura semelhante à seguinte:

Título do projeto

Apresente o nome correto da solução.

Visão geral

Explique de maneira objetiva:

- O que é o projeto;
- Qual problema ele resolve;
- Para quem ele foi desenvolvido;
- Qual é o resultado esperado;
- Como o dashboard contribui para a análise de churn e retenção.

Principais funcionalidades

Liste somente funcionalidades realmente existentes.

Exemplos que devem ser confirmados no código:

- Importação dos arquivos CSV;
- Consolidação dos dados;
- Criação do banco SQLite;
- Cruzamento de informações entre tabelas;
- Dashboard interativo;
- Indicadores de churn;
- Indicadores de retenção;
- Receita perdida;
- Contas em risco;
- Análise de uso do produto;
- Análise de tickets de suporte;
- Filtros por período, segmento, plano ou status;
- Modelo preditivo;
- Score de risco;
- Exportação de informações;

- APIs ou endpoints.

Para cada funcionalidade relevante, apresente uma breve explicação.

Tecnologias utilizadas

Identifique no repositório e documente:

- Linguagem;
- Framework web;
- Bibliotecas de dados;
- Bibliotecas de machine learning;
- Banco de dados;
- Frontend;
- Templates;
- Gráficos;
- Servidor;
- Gerenciador de dependências;
- Outras tecnologias importantes.

Não inclua tecnologias que não estejam realmente sendo utilizadas.

Arquitetura da solução

Explique o fluxo completo da aplicação.

Exemplo conceitual:

```
Arquivos CSV
↓
Validação e tratamento
↓
Carga no banco SQLite
↓
Consultas e cruzamentos
↓
Cálculo de indicadores e modelo
↓
API ou backend Python
↓
Dashboard HTML, CSS e JavaScript
```

Adapte o fluxo ao funcionamento real.

Explique também:

- Como os módulos se comunicam;
- Onde ficam as regras de negócio;
- Onde ficam as consultas ao banco;
- Onde o HTML é criado;

- Como o frontend recebe os dados;
- Como os gráficos são alimentados;
- Como o banco é inicializado.

Estrutura de diretórios

Apresente uma árvore simplificada do projeto.

Exemplo:

```
projeto/  
├─ app/  
├─ database/  
├─ data/  
├─ templates/  
├─ static/  
├─ scripts/  
├─ tests/  
├─ requirements.txt  
├─ README.md  
└─ ...
```

Para cada diretório ou arquivo relevante, explique sua responsabilidade.

Não inclua arquivos temporários, caches ou itens sem relevância para a execução.

Pré-requisitos

Documente os requisitos necessários para execução, como:

- Sistema operacional suportado;
- Versão do Python;
- Git;
- Navegador;
- Dependências;
- Permissões de acesso;
- Espaço necessário;
- Outros softwares obrigatórios.

Confirme a versão do Python e das dependências diretamente no projeto.

Instalação

Crie um passo a passo completo para alguém que acabou de clonar o repositório.

Inclua, conforme aplicável:


```
git clone URL_DO_REPOSITORIO
cd NOME_DO_PROJETO
```

Criação do ambiente virtual no Windows:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Criação do ambiente virtual no Linux ou macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

Instalação das dependências:

```
pip install -r requirements.txt
```

Utilize os comandos reais do projeto.

Caso exista `pyproject.toml`, `poetry`, `uv`, `pipenv` ou outro gerenciador, documente o procedimento correto.

Configuração

Explique todas as configurações necessárias.

Inclua, quando existir:

- Arquivo `.env`;
- Variáveis de ambiente;
- Caminho dos arquivos CSV;
- Caminho do banco SQLite;
- Porta da aplicação;
- Host;
- Configurações de segurança;
- Modo de desenvolvimento;
- Modo de produção.

Crie ou atualize um arquivo `.env.example` sem inserir senhas, tokens ou informações sensíveis.

Para cada variável, informe:

- Nome;
- Finalidade;
- Exemplo seguro;
- Se é obrigatória ou opcional.

Preparação dos dados

Explique:

- Quais arquivos de entrada são necessários;
- Onde eles devem ser colocados;
- Quais nomes ou padrões de nome são esperados;
- Quais colunas são obrigatórias;
- Como os arquivos se relacionam;
- Como os dados são tratados;
- Como valores ausentes são tratados;
- Como duplicidades são tratadas;
- Como datas são convertidas;
- Como identificadores são relacionados;
- Quais validações são realizadas.

Inclua uma tabela de arquivos de entrada semelhante a:

Arquivo	Finalidade	Chave principal	Relacionamentos
accounts.csv	Cadastro de contas	account_id	Assinaturas, uso, suporte e churn
subscriptions.csv	Dados de assinatura	subscription_id	Conta
churn_events.csv	Eventos de churn	event_id	Conta ou assinatura
feature_usage.csv	Uso do produto	usage_id	Conta
support_tickets.csv	Tickets de suporte	ticket_id	Conta

Adapte os nomes, campos e relacionamentos aos dados reais.

Criação do banco de dados

Explique como gerar o arquivo SQLite a partir dos CSVs.

Inclua:

- Script responsável;
- Diretório de entrada;
- Diretório de saída;
- Nome do banco;
- Comando para execução;
- Tabelas criadas;
- Ordem da carga;
- Regras de limpeza;
- Índices;
- Chaves;
- Relacionamentos;
- Validações realizadas.

Exemplo, somente se corresponder ao projeto:

```
python scripts/create_database.py
```

Informe claramente se o banco é:

- Criado automaticamente;
- Recriado a cada execução;
- Atualizado incrementalmente;
- Mantido no repositório;
- Ignorado pelo Git.

Execução do projeto

Apresente o comando exato para iniciar a aplicação.

Exemplos que devem ser substituídos pelo comando real:

```
python app.py
```

ou:

```
uvicorn app.main:app --reload
```

Informe:

- URL local;
- Porta;
- Página inicial;
- Como encerrar o servidor;
- O que deve aparecer no terminal;
- Como confirmar que a aplicação iniciou corretamente.

Como utilizar o dashboard

Crie um guia de uso do dashboard.

Explique:

- Como acessar;
- Qual é a página inicial;
- Quais indicadores aparecem;
- Como aplicar filtros;
- Como interpretar os gráficos;
- Como navegar entre as seções;
- Como visualizar contas em risco;
- Como interpretar o score de churn;
- Como analisar receita perdida;
- Como consultar uso do produto;
- Como consultar chamados de suporte;

- Como limpar filtros;
- Como atualizar os dados.

Para cada indicador importante, documente:

- Nome;
- Definição;
- Fórmula ou lógica;
- Fonte dos dados;
- Interpretação;
- Limitações.

Modelo preditivo de churn

Caso exista um modelo preditivo, documente detalhadamente:

- Objetivo do modelo;
- Unidade de análise;
- Variável-alvo;
- Definição de churn;
- Horizonte de previsão;
- Variáveis utilizadas;
- Variáveis descartadas;
- Preparação dos dados;
- Divisão entre treino e teste;
- Algoritmo;
- Hiperparâmetros;
- Métricas;
- Validação;
- Tratamento de desbalanceamento;
- Risco de vazamento de dados;
- Como o score é calculado;
- Faixas de risco;
- Como interpretar a previsão;
- Limitações;
- Como treinar novamente;
- Como executar somente a inferência.

Inclua comandos reais para treinamento e execução, quando existirem.

Não apresente o modelo como causal. Deixe claro que uma previsão de churn representa probabilidade ou risco estimado, conforme a implementação.

Regras de negócio

Documente as principais regras identificadas no código, incluindo:

- Definição de cliente ativo;
- Definição de churn;
- Definição de receita perdida;
- Definição de receita em risco;
- Cálculo de retenção;

- Cálculo de churn rate;
- Critérios de conta em risco;
- Segmentações;
- Tratamento de cancelamentos;
- Tratamento de planos;
- Tratamento de datas;
- Tratamento de receita recorrente;
- Regras do score de risco.

Apresente as fórmulas utilizadas sempre que possível.

Banco de dados

Inclua uma visão geral das tabelas.

Para cada tabela relevante, informe:

- Nome;
- Finalidade;
- Chave primária;
- Principais colunas;
- Relacionamentos;
- Origem dos dados.

Inclua um diagrama Mermaid, quando a estrutura permitir:

```
erDiagram
    ACCOUNTS ||--o{ SUBSCRIPTIONS : possui
    ACCOUNTS ||--o{ FEATURE_USAGE : utiliza
    ACCOUNTS ||--o{ SUPPORT_TICKETS : abre
    ACCOUNTS ||--o{ CHURN_EVENTS : gera
```

Adapte totalmente ao esquema real.

API

Caso exista API, documente:

- URL base;
- Rotas;
- Método HTTP;
- Parâmetros;
- Filtros;
- Corpo das requisições;
- Estrutura das respostas;
- Códigos de status;
- Possíveis erros.

Exemplo de estrutura:

Método	Rota	Finalidade
GET	/api/kpis	Retorna indicadores gerais
GET	/api/accounts-risk	Retorna contas em risco

Inclua exemplos reais de requisição e resposta, sem expor dados sensíveis.

Testes e validação

Documente:

- Como executar os testes;
- Tipos de teste existentes;
- O que é validado;
- Dependências de teste;
- Como interpretar os resultados.

Exemplo:

```
pytest
```

Caso não existam testes automatizados, registre isso claramente como uma limitação do projeto.

Não afirme que existem testes caso eles não estejam no repositório.

Solução de problemas

Crie uma seção de troubleshooting contendo os erros mais prováveis.

Inclua situações como:

- Python não reconhecido;
- Ambiente virtual não ativado;
- Dependência ausente;
- Porta ocupada;
- Arquivo CSV não encontrado;
- Coluna obrigatória ausente;
- Erro de encoding;
- Banco SQLite bloqueado;
- Banco não criado;
- Caminho inválido;
- Erro ao iniciar a API;
- Dashboard sem dados;
- Gráfico sem carregamento;
- Erro de importação de módulo;
- Modelo preditivo não encontrado.

Para cada problema, informe:

- Sintoma;
- Possível causa;
- Forma de correção.

Limitações conhecidas

Registre de maneira transparente:

- Funcionalidades ainda não implementadas;
- Dependência de arquivos locais;
- Restrições dos dados;
- Limitações estatísticas;
- Limitações do modelo;
- Ausência de autenticação, se aplicável;
- Ausência de atualização automática, se aplicável;
- Limitações de escalabilidade;
- Pontos que exigem evolução para produção.

Próximas evoluções

Apresente sugestões coerentes com o projeto, sem alterar o código nesta tarefa.

Exemplos:

- Automatização da ingestão de dados;
- Migração para PostgreSQL;
- Autenticação;
- Controle de acesso;
- Deploy;
- Monitoramento;
- Testes automatizados;
- Pipeline de machine learning;
- Versionamento do modelo;
- Alertas para contas em risco;
- Integração com CRM;
- Exportação de relatórios;
- Explicabilidade do modelo;
- Histórico de previsões.

Separe claramente o que já existe do que é apenas sugestão futura.

Contribuição

Explique um fluxo básico de contribuição:

- Criar branch;
- Implementar alteração;
- Executar testes;
- Atualizar documentação;

- Criar commit;
- Abrir pull request.

Licença

Verifique se existe licença no repositório.

Se existir, documente-a.

Se não existir, não escolha uma licença automaticamente. Apenas informe que o projeto ainda não possui uma licença definida.

2. docs/ARCHITECTURE.md

Crie um documento específico de arquitetura contendo:

- Visão geral técnica;
- Componentes;
- Fluxo de dados;
- Backend;
- Frontend;
- Banco;
- Camada de dados;
- Camada de serviços;
- API;
- Modelo preditivo;
- Dependências entre módulos;
- Decisões arquiteturais;
- Limitações;
- Diagramas Mermaid.

Inclua pelo menos:

Diagrama de componentes

```
flowchart LR
    CSV[Arquivos CSV] --> ETL[Processamento e validação]
    ETL --> DB[(SQLite)]
    DB --> API[Backend Python]
    API --> DASH[Dashboard Web]
```

Fluxo de execução

Mostre o que acontece desde a leitura dos dados até a exibição no dashboard.

Decisões técnicas

Explique, com base no projeto:

- Por que o SQLite é adequado para o protótipo;

- Como o Python centraliza a solução;
- Como o HTML, CSS e JavaScript são utilizados;
- Como os dados chegam aos gráficos;
- Quais pontos precisariam mudar para produção.

3. docs/DATABASE.md

Crie uma documentação específica do banco contendo:

- Visão geral;
- Arquivo físico;
- Localização;
- Processo de criação;
- Tabelas;
- Colunas;
- Tipos;
- Chaves;
- Índices;
- Relacionamentos;
- Consultas importantes;
- Regras de integridade;
- Processo de atualização;
- Processo de recriação;
- Backup;
- Limitações.

Inclua um diagrama entidade-relacionamento em Mermaid.

4. docs/DATA_DICTIONARY.md

Crie um dicionário de dados.

Organize por arquivo ou tabela.

Utilize uma estrutura semelhante a:

Campo	Tipo	Obrigatório	Descrição	Exemplo	Regra
-------	------	-------------	-----------	---------	-------

Documente somente campos identificados no código, banco ou arquivos de dados.

5. docs/MODEL.md

Caso exista modelo preditivo, crie uma documentação exclusiva contendo:

- Problema;
- Definição do target;
- Features;
- Pré-processamento;
- Algoritmo;

- Treinamento;
- Validação;
- Métricas;
- Score;
- Faixas de risco;
- Interpretação;
- Limitações;
- Possíveis vieses;
- Risco de data leakage;
- Reprodutibilidade;
- Retreinamento;
- Monitoramento futuro.

Caso não exista modelo preditivo implementado, não invente essa documentação. Em vez disso, crie uma seção informando que a análise atual não possui um modelo preditivo operacional e explique quais componentes seriam necessários para implementá-lo.

6. docs/USER_GUIDE.md

Crie um manual de utilização destinado a usuários não técnicos.

Evite excesso de termos técnicos.

Explique:

- O que é o dashboard;
- Como acessar;
- O que cada seção apresenta;
- Como utilizar filtros;
- Como interpretar KPIs;
- Como interpretar contas em risco;
- Como usar as análises na tomada de decisão;
- Cuidados na interpretação;
- Limitações dos dados.

7. docs/TROUBLESHOOTING.md

Crie um guia detalhado de erros e soluções.

Inclua comandos de diagnóstico quando forem seguros e úteis.

8. .env.example

Caso o projeto use variáveis de ambiente, crie ou atualize o `.env.example`.

Nunca coloque:

- Senhas reais;
- Tokens;
- Chaves;

- Dados pessoais;
- Credenciais;
- Caminhos pessoais do desenvolvedor.

9. requirements.txt

Analise as importações do projeto e confira o arquivo de dependências.

Faça apenas os ajustes necessários para que:

- Todas as bibliotecas utilizadas estejam declaradas;
- Dependências claramente não utilizadas sejam avaliadas;
- Não existam pacotes duplicados;
- O projeto possa ser instalado do zero.

Não atualize versões indiscriminadamente.

Evite quebrar dependências já validadas.

10. .gitignore

Confirme se o `.gitignore` contempla, conforme aplicável:

```
.venv/  
__pycache__/  
*.pyc  
.env  
.pytest_cache/  
.vscode/  
.idea/  
.DS_Store  
Thumbs.db  
*.log
```

Avalie cuidadosamente se o banco SQLite e os arquivos CSV devem ser ignorados.

Não ignore dados necessários para demonstração sem antes entender a estrutura do projeto.

Qualidade da documentação

A documentação deve:

- Ser escrita em português do Brasil;
- Ser clara para pessoas técnicas e não técnicas;
- Utilizar Markdown;
- Possuir títulos e subtítulos bem organizados;
- Usar tabelas quando facilitarem a compreensão;
- Usar blocos de código para comandos;

- Usar Mermaid para diagramas;
- Ser objetiva, mas suficientemente detalhada;
- Evitar textos genéricos;
- Não conter afirmações não confirmadas;
- Não expor dados sensíveis;
- Não incluir caminhos pessoais do Windows na documentação final;
- Utilizar caminhos relativos sempre que possível;
- Separar o que está implementado do que é melhoria futura.

Validação obrigatória

Após criar a documentação:

1. Execute os comandos de instalação descritos, quando o ambiente permitir;
2. Confirme que o comando de inicialização está correto;
3. Confirme a URL e a porta utilizadas;
4. Verifique se todos os caminhos citados existem;
5. Verifique se os nomes dos arquivos estão corretos;
6. Verifique se todas as dependências necessárias estão declaradas;
7. Verifique se os diagramas Mermaid correspondem à estrutura real;
8. Confirme que nenhuma senha ou credencial foi inserida;
9. Confirme que nenhuma funcionalidade inexistente foi documentada;
10. Confira se uma pessoa que nunca viu o projeto conseguiria executá-lo apenas seguindo o README.

Restrições

Nesta tarefa, não altere regras de negócio nem modifique o comportamento funcional da aplicação.

Não refatore a solução, salvo pequenos ajustes indispensáveis para corrigir inconsistências diretamente relacionadas à documentação, como:

- Comando de inicialização incorreto;
- Dependência ausente;
- Arquivo de exemplo inexistente;
- Caminho fixo que impeça a execução em outro computador.

Caso seja necessário alterar código, faça mudanças mínimas e explique exatamente:

- O que foi alterado;
- Por que foi necessário;
- Quais arquivos foram modificados;
- Qual impacto a alteração possui.

Resultado esperado

Ao final, apresente um resumo contendo:

1. Documentos criados;

2. Documentos atualizados;
3. Estrutura final da documentação;
4. Comando correto para instalar o projeto;
5. Comando correto para gerar ou atualizar o banco;
6. Comando correto para executar a aplicação;
7. URL local do dashboard;
8. Dependências corrigidas;
9. Inconsistências encontradas;
10. Limitações identificadas;
11. Alterações realizadas no código, caso existam;
12. Pontos que ainda precisam de decisão humana.

A documentação final deve transformar o repositório em um projeto que possa ser compreendido, instalado, executado, demonstrado e futuramente mantido por outro desenvolvedor sem depender de conhecimento verbal do autor.

Registro complementar do processo no Codex

Esta seção complementa o Process Log da submissão com evidências da implementação, validação dos cálculos, reformulação visual e correção do carregamento dos dados.

Etapa	Objetivo	Resultado
1º prompt	Construir o protótipo Flask conectado ao SQLite	Arquitetura modular, APIs, dashboard, exploração de contas, exportação CSV, README e testes.
2º prompt	Explicar como os KPIs e métricas foram calculados	Documentação da assinatura representativa, definição consolidada de churn e fórmulas dos cards.
3º prompt	Reformular o layout para uma apresentação executiva	Novo shell SaaS, componentes visuais, responsividade, acessibilidade e preservação das APIs.
4º prompt	Corrigir gráficos que permaneciam vazios	Primeira tentativa com fallback local para indisponibilidade do Plotly/CDN.
5º prompt	Garantir a exibição dos dados mesmo sem Plotly	Renderização local prioritária, cache-busting e validação com base completa de 500 contas.

Etapa 1 - Resultado do primeiro prompt: implementação do protótipo

O primeiro prompt resultou na criação do protótipo Flask conectado ao banco SQLite. O retorno registra a arquitetura modular, os endpoints, os arquivos criados e alterados, os testes executados e as instruções de inicialização.

Evidência 1.1 - Implementação, arquitetura e validação inicial

Implementado o protótipo Flask completo em:

`C:\Users\Usuario\Desktop\Github\ai-master-challenge\submissions\bruno-milford\solution`

Criei a arquitetura modular pedida, com validação do SQLite, APIs JSON, dashboard Jinja, JS com `fetch`, gráficos Plotly, tabela de risco, exploração de contas, detalhe por conta, exportação CSV, README e testes.

Arquivos principais criados/alterados

- `app.py`
- `config.py`
- `requirements.txt`
- `README.md`
- `services/` : banco, dashboard, churn, risco, contas
- `routes/` : rotas de página e API
- `templates/` : dashboard, contas e detalhe
- `static/css/styles.css`
- `static/js/dashboard.js` , `accounts.js` , `account_detail.js`
- `tests/test_app.py`

Arquitetura implementada

- Flask + SQLite com consultas somente leitura.
- Caminho do banco resolvido via `pathlib` , relativo ao projeto.
- Validação de existência do banco e das 5 tabelas no boot.
- KPIs e gráficos calculados a partir do SQLite real.
- Assinatura considerada por conta: prioriza assinatura sem `end_date` ; se não houver, usa a mais recente por `start_date` .
- Score heurístico documentado em constantes Python em `services/risk_service.py`.
- Exportação CSV em `/api/export/risk-accounts.csv` .

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 1.2 - Implementação, arquitetura e validação inicial

Endpoints disponíveis

- `/api/health`
- `/api/filters`
- `/api/kpis`
- `/api/churn/timeline`
- `/api/churn/reasons`
- `/api/churn/segments`
- `/api/revenue`
- `/api/usage`
- `/api/support`
- `/api/reactivation`
- `/api/risk-accounts`
- `/api/accounts`
- `/api/accounts/<account_id>`
- `/api/export/risk-accounts.csv`

Testes realizados

- `unittest` : 7 testes passaram.
- Validação HTTP local via servidor Werkzeug:
 - `/api/health` : 200
 - `/api/kpis` : 200 , retornando 500 contas
 - `/api/export/risk-accounts.csv` : 200
 - `/` : 200
 - `/accounts` : 200
 - `/accounts/A-2e4581` : 200
 - `/api/accounts/A-2e4581` : 200

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 1.3 - Implementação, arquitetura e validação inicial

Como executar

```
powershell

cd "C:\Users\Usuario\Desktop\Github\ai-master-challenge\submissions\bruno-milford\solution"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python app.py
```

Depois abra:

`http://127.0.0.1:5000`

Limitações conhecidas

- Nesta sessão, o comando `python` não estava no PATH e a `.venv` existente apontava para um Python sem acesso; validei com o Python embutido do pgAdmin.
- Plotly.js e Bootstrap estão carregados por CDN na interface. Uma próxima evolução boa seria vendorizá-los localmente para cumprir uso totalmente offline.
- O score é heurístico e explicável, não modelo preditivo calibrado.

 **README.md**
Documento · MD

Abrir ▾

24 arquivos alterados +1959 -7

Desfazer ↶ Revisar ↗

config.py +16 -0 •

routes/__init__.py +1 -0 •

services/__init__.py +1 -0 •

services/churn_service.py +128 -0 •

services/dashboard_service.py +395 -0 •

services/database_service.py +90 -0 •

services/account_service.py +107 -0 •

↓

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 1.4 - Implementação, arquitetura e validação inicial

config.py	+16 -0	•	▼
routes/__init__.py	+1 -0	•	▼
services/__init__.py	+1 -0	•	▼
services/chum_service.py	+128 -0	•	▼
services/dashboard_service.py	+395 -0	•	▼
services/database_service.py	+90 -0	•	▼
services/account_service.py	+107 -0	•	▼
services/risk_service.py	+201 -0	•	▼
app.py	+47 -0	•	▼
routes/api_routes.py	+131 -0	•	▼
routes/page_routes.py	+26 -0	•	▼
templates/account_detail.html	+31 -0	•	▼
templates/accounts.html	+19 -0	•	▼
templates/base.html	+42 -0	•	▼
templates/dashboard.html	+77 -0	•	▼
static/css/styles.css	+206 -0	•	▼
static/js/account_detail.js	+38 -0	•	▼
static/js/accounts.js	+26 -0	•	▼
static/js/dashboard.js	+164 -0	•	▼
README.md	+117 -0	•	▼
requirements.txt	+2 -0	•	▼
tests/test_app.py	+69 -0	•	▼
services/database_service.py	+21 -7		▼

Captura de tela do processo e do retorno produzido pelo Codex.

Etapa 2 - Explicação dos cálculos e das métricas

Após a implementação, foi solicitado ao Codex que explicasse a origem e a lógica dos indicadores exibidos. A explicação foi incorporada à metodologia abaixo para tornar os números auditáveis e evitar dupla contagem de assinaturas.

Métrica	Regra de cálculo
Assinatura representativa por conta	Prioriza uma assinatura sem end_date. Se não existir assinatura ativa, utiliza a assinatura mais recente por start_date. Essa regra evita somar várias assinaturas históricas da mesma conta.
Total de contas	COUNT(*) da base consolidada de contas após a aplicação dos filtros.
Contas ativas	Contas em que churned_account = 0.
Contas com churn	Contas em que churned_account = 1.
Taxa de churn	Contas com churn / total de contas x 100.
MRR ativo	Soma de mrr_amount da assinatura representativa somente para contas ativas.
ARR ativo	Soma de arr_amount da assinatura representativa somente para contas ativas.
MRR perdido	Soma de mrr_amount da assinatura representativa das contas classificadas com churn.
ARR perdido	Soma de arr_amount da assinatura representativa das contas classificadas com churn.
Ticket médio mensal	Média do mrr_amount das contas cujo MRR é maior que zero.
Total de tickets	Soma dos tickets de suporte associados às contas filtradas.
Contas reativadas	Contas com ao menos um evento em churn_events em que is_reactivation = 1.
Alto risco	Contas ativas cujo risk_score é maior ou igual a 60.

Definição consolidada de churn: se o evento mais recente for uma reativação, a conta é considerada ativa. Caso contrário, a conta entra como churn quando existe evento efetivo de churn ou quando o cadastro possui churn_flag = 1.

Exemplo validado: 500 contas, 350 contas com churn, taxa de churn de 70,00% e 150 contas ativas. A receita ativa é calculada sobre as 150 contas ativas, enquanto MRR e ARR perdidos são calculados sobre as 350 contas com churn.

Evidência 2.1 - Explicação da lógica e das fórmulas



Como voce fez o calculo desses dados ?

Trabalhou por 26s >

O cálculo desses cards vem do endpoint `/api/kpis`, implementado em `services/dashboard_service.py` (line 93) e complementado em `routes/api_routes.py` (line 42).

A regra mais importante é: eu **não** **somo** **todas** **as** **assinaturas** **históricas**. Primeiro monto uma base consolidada de conta, escolhendo **uma assinatura representativa por conta**:

1. Se a conta tem assinatura sem `end_date`, ela tem prioridade.
2. Se não tem assinatura ativa, uso a assinatura mais recente por `start_date`.
3. Isso evita inflar MRR/ARR somando várias assinaturas antigas da mesma conta.

Essa regra está no CTE `ranked_subscriptions`, em `services/dashboard_service.py` (line 344).

Os cards da imagem foram calculados assim:

Card	Cálculo
Total de contas	<code>COUNT(*)</code> da base consolidada de contas filtradas
Contas ativas	contas onde <code>churned_account = 0</code>
Contas com churn	contas onde <code>churned_account = 1</code>
Taxa de churn	<code>contas com churn / total de contas * 100</code>
MRR ativo	soma do <code>mrr_amount</code> da assinatura considerada apenas para contas ativas
ARR ativo	soma do <code>arr_amount</code> da assinatura considerada apenas para contas ativas
MRR perdido	soma do <code>mrr_amount</code> da assinatura considerada para contas com churn
ARR perdido	soma do <code>arr_amount</code> da assinatura considerada para contas com churn

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 2.2 - Explicação da lógica e das fórmulas

Card	Cálculo
Total de contas	<code>COUNT(*)</code> da base consolidada de contas filtradas
Contas ativas	contas onde <code>churned_account = 0</code>
Contas com churn	contas onde <code>churned_account = 1</code>
Taxa de churn	<code>contas com churn / total de contas * 100</code>
MRR ativo	soma do <code>mrr_amount</code> da assinatura considerada apenas para contas ativas
ARR ativo	soma do <code>arr_amount</code> da assinatura considerada apenas para contas ativas
MRR perdido	soma do <code>mrr_amount</code> da assinatura considerada para contas com churn
ARR perdido	soma do <code>arr_amount</code> da assinatura considerada para contas com churn
Ticket médio mensal	média do <code>mrr_amount</code> das contas com MRR maior que zero
Total de tickets	soma dos tickets de suporte das contas filtradas
Contas reativadas	contas com pelo menos um evento em <code>churn_events</code> onde <code>is_reactivation = 1</code>
Alto risco	contas ativas com <code>risk_score >= 60</code>

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 2.3 - Explicação da lógica e das fórmulas

A definição de churn consolidado é esta:

```
sql

CASE
  WHEN ultimo_evento.is_reactivation = 1 THEN 0
  WHEN ultimo_evento.churn_event_id IS NOT NULL THEN 1
  WHEN accounts.churn_flag = 1 THEN 1
  ELSE 0
END AS churned_account
```

Ou seja: se o evento mais recente da conta for reativação, ela fica como ativa. Caso contrário, se houver evento de churn ou `churn_flag`, ela entra como churn.

No exemplo da imagem:

```
text

Total de contas = 500
Contas com churn = 350
Taxa de churn = 350 / 500 = 70.00%
Contas ativas = 500 - 350 = 150
```

E a receita:

```
text

MRR ativo = soma do MRR das 150 contas ativas
MRR perdido = soma do MRR das 350 contas churn
ARR ativo = soma do ARR das contas ativas
ARR perdido = soma do ARR das contas churn
```

O "Alto risco = 1" vem do score heurístico em [services/risk_service.py](#): ele conta somente contas **ativas** com `risk_score >= 60`.

Captura de tela do processo e do retorno produzido pelo Codex.

Etapa 3 - Prompt de reformulação visual

O terceiro prompt solicitou a reformulação do dashboard para um produto SaaS executivo, mantendo Flask, Jinja2, SQLite, endpoints, consultas e regras existentes. O prompt completo é reproduzido ao final desta seção.

Evidência 3.1 - Resultado da auditoria e do ajuste de layout

Concluído. Reestruturei a interface mantendo a logica, endpoints, consultas SQL e regras de calculo intactas.

Auditoria Realizada

- Páginas existentes: `/` , `/accounts` , `/accounts/<account_id>` .
- Templates reutilizados: `base.html` , `dashboard.html` , `accounts.html` , `account_detail.html` .
- IDs preservados para JS: `filtersForm` , `kpiGrid` , `churnTimeline` , `riskTable` , `accountsTable` , `accountFacts` , etc.
- Endpoints consumidos preservados: `/api/kpis` , `/api/risk-accounts` , `/api/accounts` , `/api/churn/*` , `/api/revenue` , `/api/usage` , `/api/support` , export CSV.
- Nenhuma regra de churn, receita ou score foi alterada.

Arquivos Criados

-  `static/js/api.js`
-  `static/js/ui.js`
-  `static/js/formatters.js`
-  `static/js/charts.js`

Arquivos Alterados

- `</>` `templates/base.html`
- `</>` `templates/dashboard.html`
- `</>` `templates/accounts.html`
- `</>` `templates/account_detail.html`
- `#` `static/css/styles.css`
-  `static/js/dashboard.js`
-  `static/js/accounts.js`
-  `static/js/account_detail.js`
-  `README.md`

Melhorias Implementadas

- Shell SaaS com sidebar escura, topbar contextual e layout responsivo.
- KPIs agrupados por Base, Receita, Experiencia e Risco.
- Filtros globais em card recolhível, chips de filtros ativos  ar mobile.
- Secao "O que exige atencao" com insights derivados das APIs existentes.

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 3.2 - Resultado da auditoria e do ajuste de layout

Concluído. Reestruturei a interface mantendo a logica, endpoints, consultas SQL e regras de calculo intactas.

Auditoria Realizada

- Paginas existentes: `/` , `/accounts` , `/accounts/<account_id>` .
- Templates reutilizados: `base.html` , `dashboard.html` , `accounts.html` , `account_detail.html` .
- IDs preservados para JS: `filtersForm` , `kpiGrid` , `churnTimeline` , `riskTable` , `accountsTable` , `accountFacts` , etc.
- Endpoints consumidos preservados: `/api/kpis` , `/api/risk-accounts` , `/api/accounts` , `/api/churn/*` , `/api/revenue` , `/api/usage` , `/api/support` , export CSV.
- Nenhuma regra de churn, receita ou score foi alterada.

Arquivos Criados

-  `static/js/api.js`
-  `static/js/ui.js`
-  `static/js/formatters.js`
-  `static/js/charts.js`

Arquivos Alterados

- `</> templates/base.html`
- `</> templates/dashboard.html`
- `</> templates/accounts.html`
- `</> templates/account_detail.html`
- `# static/css/styles.css`
-  `static/js/dashboard.js`
-  `static/js/accounts.js`
-  `static/js/account_detail.js`
-  `README.md`

Melhorias Implementadas

- Shell SaaS com sidebar escura, topbar contextual e layout responsivo.
- KPIs agrupados por Base, Receita, Experiencia e Risco.
- Filtros globais em card recolhivel, chips de filtros ativos  awer mobile.
- Secao "O que exige atencao" com insights derivados das APIs existentes.

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 3.3 - Resultado da auditoria e do ajuste de layout

Melhorias Implementadas

- Shell SaaS com sidebar escura, topbar contextual e layout responsivo.
- KPIs agrupados por Base, Receita, Experiencia e Risco.
- Filtros globais em card recolhível, chips de filtros ativos e drawer mobile.
- Secao "O que exige atencao" com insights derivados das APIs existentes.
- Graficos Plotly com configuracao centralizada e visual mais limpo.
- Tabela de risco com resumo por classificacao, filtros rapidos e colunas mais executivas.
- `/accounts` reformulada com busca, filtros rapidos, risco, status e ultima atividade.
- Detalhe de conta reformulado como visao de Customer Success com cards, abas e timeline visual.
- Skeleton loaders, estados vazios, toasts e mensagens de erro mais profissionais.
- Melhor foco visivel, labels, aria-labels e botoes mais acessiveis.

Validação

- `unittest` : 7 testes passaram.
- `node --check` em todos os arquivos JS: passou.
- Smoke HTTP:
 - `/` : 200
 - `/accounts` : 200
 - `/accounts/<id>` : 200
 - `/api/health` : 200
 - `/api/kpis` : 200
 - `/api/churn/timeline` : 200
 - `/api/support` : 200
 - `/api/risk-accounts` : 200
 - `/api/export/risk-accounts.csv` : 200
- Chrome headless gerou capturas desktop/mobile e confirmou renderizacao visual com dados reais.

Limitação

- O navegador integrado do Codex bloqueou `localhost/127.0.0.1` com `ERR_BLOCKED_BY_CLIENT` ; usei Chrome headless local como fallback para validacao visual.



Captura de tela do processo e do retorno produzido pelo Codex.

Etapa 4 - Primeira tentativa de correção do carregamento

Mesmo após a reformulação visual, os gráficos podiam permanecer vazios quando o Plotly carregado por CDN falhava ou demorava. A primeira correção tornou o carregamento assíncrono e adicionou um fallback local.

Evidência 4.1 - Tentativa inicial de correção dos gráficos vazios

Como executar

```
powershell  
  
cd "C:\Users\Usuario\Desktop\Github\ai-master-challenge\submissions\bruno-milford\solution"  
python app.py
```

Acesse:

<http://127.0.0.1:5000>

 **README.md**
Documento · MD

Abrir ▾

17 arquivos alterados **+1766 -18**

Desfazer ↶Revisar ↷⌵

templates/base.html	+76 -0 ●	▾
templates/dashboard.html	+159 -0 ●	▾
templates/account_detail.html	+74 -0 ●	▾
templates/accounts.html	+45 -0 ●	▾
static/css/styles.css	+537 -0 ●	▾
static/js/api.js	+15 -0 ●	▾
static/js/charts.js	+119 -0 ●	▾
static/js/dashboard.js	+362 -0 ●	▾
static/js/formatters.js	+31 -0 ●	▾
static/js/ui.js	+70 -0 ●	▾
static/js/account_detail.js	+102 -0 ●	▾
static/js/accounts.js	+137 -0 ●	▾
_run_server_tmp.py	+8 -0 ●	▾

↓

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 4.2 - Tentativa inicial de correção dos gráficos vazios



Não está carregando os dados, mesmo aplicando os filtros e assim, caso não tiver filtro nenhum, pode carregar todos os dados

Trabalhou por 3m 7s >

Corrigido. O problema era que os gráficos dependiam do `Plotly.js` via CDN. Quando a CDN não carregava ou demorava, os containers ficavam vazios, mesmo com a API retornando dados.

O que ajustei:

- Mantive o carregamento do Plotly, mas ele agora é `async` e não bloqueia os scripts locais.
- Adicionei fallback local em `static/js/charts.js`: se `Plotly` não estiver disponível, os gráficos renderizam como barras/linhas HTML/CSS usando os mesmos dados reais da API.
- Adicionei estilos dos gráficos fallback em `# static/css/styles.css`.
- Mantive o comportamento correto: sem filtro nenhum, carrega a base completa.
- Atualizei o README mencionando o fallback.

Validei:

- `/api/kpis` retorna `500` contas sem filtro.
- `/api/churn/timeline`, `/api/churn/reasons`, `/api/churn/segments` retornam `200`.
- Testei com `cdn.plot.ly` bloqueado e os gráficos renderizaram com `fallback-chart`.
- `unittest` : 7 testes passaram.
- `node --check` nos JS alterados: passou.

Agora pode rodar normalmente:

```
powershell  
python app.py
```

E abrir: 

Captura de tela do processo e do retorno produzido pelo Codex.

Etapa 5 - Correção bem-sucedida da exibição dos dados

Como a primeira tentativa ainda não resolveu integralmente o problema, foi solicitado novo ajuste. A solução passou a renderizar primeiro um gráfico local em HTML/CSS com os dados reais da API e, quando disponível, o Plotly aprimora a visualização. Também foram adicionados cache-busting e tratamento para falhas de APIs secundárias.

Evidência 5.1 - Correção final e validação do carregamento

E abrir:

text

http://127.0.0.1:5000

README.md

Documento · MD

Abrir

4 arquivos alterados +149 -1

Desfazer ↶ Revisar ↷

static/js/charts.js +78 -0

static/css/styles.css +69 -0

templates/base.html +1 -1

README.md +1 -0

Não deu certo

Os dados tem que aparecer, como vou apresentar uma análise de dados sem dados

Trabalhou por 11m 59s >

Corrigi de forma mais robusta agora.

O problema era: os dados vinham da API, mas os gráficos podiam ficar em branco quando o Plotly/CDN falhava ou demorava. Agora os gráficos **sempre mostram dados**, mesmo sem Plotly.

O que mudei:

static/js/charts.js

↓

Renderiza primeiro um gráfico local HTML/CSS com os dados reais.

Captura de tela do processo e do retorno produzido pelo Codex.

Submissão - Bruno Milford - Evidências complementares do processo com IA

Página adicional 15

Evidência 5.2 - Correção final e validação do carregamento

O que mudei:

- `static/js/charts.js`
 - Renderiza primeiro um gráfico local HTML/CSS com os dados reais.
 - Se o Plotly estiver disponível, ele melhora o gráfico depois.
 - Se o Plotly falhar, o gráfico local continua visível.
 - Nada fica mais em branco.
- `templates/base.html`
 - Adicionei cache-busting nos arquivos CSS/JS para o navegador não usar versão antiga.
- `static/js/dashboard.js`
 - O dashboard agora não derruba tudo se uma API secundária falhar.
 - Sem filtro nenhum, continua carregando a base completa.

Validei com a CDN do Plotly bloqueada e os dados apareceram nos gráficos:

- Churn ao longo do tempo
- Motivos de churn
- Churn por plano
- Churn por indústria
- Churn por país
- KPIs com 500 contas

Também rodei:

- `unittest` : passou
- `node --check` : passou
- APIs principais: 200



Agora rodei:

Pedir alterações adicionais

Captura de tela do processo e do retorno produzido pelo Codex.

Evidência 5.3 - Correção final e validação do carregamento

Também rodei:

- `unittest` : passou
- `node --check` : passou
- APIs principais: `200`

Agora rode:

```
powershell  
python app.py
```

Abra:

```
text  
  
http://127.0.0.1:5000
```

Se ainda aparecer vazio no navegador, faça um hard refresh: `Ctrl + F5` , porque antes ele podia estar segurando JS antigo em cache.

**Prévia da web**
Site Abrir

6 arquivos alterados +144 -81	Desfazer ↶	Revisar ↗	⌘
templates/account_detail.html +2 -2			▼
templates/accounts.html +2 -2			▼
templates/base.html +10 -10			▼
templates/dashboard.html +2 -2			▼
static/js/charts.js +104 -64			▼
static/js/dashboard.js +24 -1			▼

    domingo, 15:18

Captura de tela do processo e do retorno produzido pelo Codex.

Apêndice C - Prompt completo de reformulação do layout

Texto enviado ao Codex para orientar a auditoria e a reformulação visual do protótipo.

Atue como um desenvolvedor frontend sênior, especialista em UX/UI para dashboards executivos, produtos SaaS B2B, visualização de dados e aplicações Flask com HTML, CSS e JavaScript.

Contexto

Já existe um dashboard interativo chamado:

```
```text
RavenStack Churn Intelligence
```
```

O sistema foi desenvolvido como um protótipo para análise de churn de uma empresa SaaS B2B.

A aplicação utiliza:

- * Python;
- * Flask;
- * SQLite;
- * Jinja2;
- * HTML5;
- * CSS3;
- * JavaScript;
- * Plotly.js;
- * Bootstrap 5, quando necessário.

O projeto está localizado em:

```
```text
C:\Users\Usuario\Desktop\Github\ai-master-challenge\submissions\bruno-milford\solution
```
```

O banco está localizado em:

```
```text
database\ravenstack.db
```
```

A aplicação já possui ou deverá possuir:

- * dashboard executivo;
- * KPIs;
- * filtros;
- * gráficos;
- * análise de churn;
- * análise de receita;
- * análise de uso do produto;
- * análise de suporte;
- * tabela de contas em risco;

- * página de exploração de contas;
- * página de detalhes de cada conta;
- * score heurístico de risco;
- * endpoints Flask que retornam dados em JSON.

Objetivo desta tarefa

Reestruture e aprimore o layout da solução para que ela tenha aparência de um produto SaaS profissional, moderno e executivo.

O foco principal desta tarefa é:

- * melhorar a experiência visual;
- * organizar melhor as informações;
- * criar uma hierarquia clara;
- * tornar o dashboard mais profissional;
- * melhorar a leitura dos KPIs e gráficos;
- * melhorar filtros, tabelas e navegação;
- * melhorar a responsividade;
- * preservar integralmente a lógica e os dados já implementados.

Não transforme a aplicação em outro framework.

Não utilize:

- * React;
- * Vue;
- * Angular;
- * Streamlit;
- * Dash;
- * bibliotecas externas pesadas de componentes;
- * serviços externos;
- * templates pagos;
- * CDN desnecessária.

Continue utilizando Flask, Jinja2, HTML, CSS e JavaScript.

Regra principal

Antes de realizar qualquer alteração:

1. inspecione toda a estrutura atual do projeto;
2. execute a aplicação;
3. identifique as páginas existentes;
4. identifique os templates e componentes reutilizados;
5. identifique os endpoints consumidos;
6. identifique os IDs e seletores utilizados pelo JavaScript;
7. entenda como filtros, gráficos e tabelas são atualizados;

8. preserve os comportamentos já funcionais;
9. não altere cálculos, regras de negócio ou consultas SQL sem necessidade;
10. não remova funcionalidades existentes.

O objetivo é reformular a interface, não reescrever a aplicação inteira.

Direção visual

O dashboard deve transmitir:

- * confiabilidade;
- * clareza;
- * inteligência analítica;
- * maturidade;
- * controle;
- * visão executiva;
- * tecnologia B2B.

O resultado deve lembrar um produto SaaS moderno de inteligência de receita e Customer Success, evitando aparência de trabalho acadêmico, template genérico ou painel administrativo antigo.

A interface deve ser predominantemente clara, com menu lateral escuro.

Identidade visual

Utilize a seguinte paleta como base:

```text

Azul-marinho principal: #0F172A  
Azul-marinho secundário: #172033  
Azul de destaque: #2563EB  
Azul hover: #1D4ED8  
Azul suave: #DBEAFE  
Azul muito suave: #EFF6FF

Verde: #16A34A  
Verde suave: #DCFCE7

Amarelo/laranja: #D97706  
Amarelo suave: #FEF3C7

Vermelho: #DC2626  
Vermelho suave: #FEE2E2

Roxo auxiliar: #7C3AED  
Roxo suave: #EDE9FE

Fundo principal: #F5F7FB

Fundo secundário: #F8FAFC  
Cards: #FFFFFF  
Bordas: #E2E8F0  
Texto principal: #0F172A  
Texto secundário: #475569  
Texto discreto: #64748B  
```

Não use cores saturadas em excesso.

Cores de alerta devem ser usadas apenas onde houver significado real.

Evite:

- * gradientes fortes;
- * sombras pesadas;
- * cards excessivamente arredondados;
- * efeitos neon;
- * animações desnecessárias;
- * excesso de ícones;
- * gráficos multicoloridos sem propósito;
- * aparência infantil.

Tipografia

Utilize preferencialmente:

```
```text
Inter
```
```

Caso não seja desejável carregar uma fonte externa, utilize uma pilha segura:

```
```css
font-family:
 Inter,
 ui-sans-serif,
 system-ui,
 -apple-system,
 BlinkMacSystemFont,
 "Segoe UI",
 sans-serif;
```
```

Defina uma hierarquia tipográfica consistente:

- * título principal: 24 a 30 px;
- * título de seção: 18 a 22 px;

- * título de card: 13 a 14 px;
- * valor de KPI: 26 a 34 px;
- * texto padrão: 14 a 16 px;
- * legenda: 12 a 13 px.

Evite textos muito pequenos.

Estrutura geral da aplicação

Crie um shell visual consistente com:

```
```text
Sidebar fixa ou recolhível
 +
Topbar
 +
Área principal de conteúdo
```
```

Sidebar

A sidebar deve ter:

- * largura aproximada de 250 a 270 px quando expandida;
- * versão recolhida em telas menores;
- * fundo azul-marinho;
- * logotipo textual ou símbolo simples;
- * nome "RavenStack";
- * identificação "Churn Intelligence";
- * itens de navegação;
- * destaque claro do item ativo;
- * agrupamento por contexto;
- * rodapé discreto.

Sugestão de navegação:

```
```text
VISÃO EXECUTIVA
- Visão Geral
```

```
ANÁLISES
- Churn
- Receita
- Produto e Uso
- Suporte
```

```
CUSTOMER SUCCESS
- Contas em Risco
```

- Explorar Contas  
```

No rodapé da sidebar, apresente discretamente:

```text  
Protótipo Analítico  
Base local SQLite  
```

Não use emojis como ícones.

Utilize ícones SVG simples ou uma biblioteca já existente no projeto. Caso não exista biblioteca de ícones, crie SVGs inline pequenos e consistentes.

Topbar

A topbar deve conter:

- * título da página atual;
- * descrição curta da página;
- * indicador de atualização;
- * ação para atualizar os dados;
- * botão para abrir filtros em telas menores;
- * menu mobile;
- * breadcrumb apenas quando ajudar na navegação.

Exemplo:

```text  
Visão Geral  
Acompanhe churn, receita, engajamento e risco da base de clientes.  
```

Conteúdo principal

A área principal deve ter:

- * largura fluida;
- * largura máxima quando necessário;
- * espaçamento lateral consistente;
- * bom uso de espaço em monitores grandes;
- * espaçamento vertical entre 20 e 32 px;
- * separação clara entre seções;
- * alinhamento consistente dos elementos.

Evite deixar grandes espaços vazios ou concentrar tudo em uma única coluna.

Cabeçalho da página inicial

No topo do dashboard, crie uma área introdutória compacta contendo:

- * título "Visão Executiva";
- * pequena descrição;
- * período analisado;
- * status dos dados;
- * quantidade de contas consideradas;
- * botão "Atualizar dados";
- * botão ou controle "Filtros".

Não crie um banner grande. O cabeçalho deve ser informativo e compacto.

Painel de filtros

Reestruture os filtros globais.

Em desktop:

- * apresente os filtros dentro de um card;
- * permita recolher e expandir;
- * organize os campos em grid;
- * use labels claras;
- * mantenha altura consistente dos inputs;
- * use selects e campos de data visualmente uniformes;
- * mostre filtros ativos;
- * disponibilize "Aplicar filtros" e "Limpar".

Em telas menores:

- * abra os filtros em drawer lateral ou painel sobreposto;
- * preserve todos os filtros;
- * permita fechar com facilidade.

Crie uma área de filtros aplicados com chips, por exemplo:

```
```text
Plano: Enterprise ×
País: Brazil ×
Status: Ativo ×
```
```

O usuário deve conseguir remover filtros individualmente quando for viável.

Não altere o formato de envio dos filtros para a API sem necessidade.

KPIs

Reestruture os cards de KPIs.

Organize-os em grupos lógicos.

Grupo 1 – Base de clientes

- * Total de contas;
- * Contas ativas;
- * Contas com churn;
- * Taxa de churn.

Grupo 2 – Receita

- * MRR ativo;
- * ARR ativo;
- * MRR perdido;
- * ARR perdido.

Grupo 3 – Experiência e risco

- * Total de tickets;
- * Satisfação média;
- * Contas reativadas;
- * Contas em alto risco.

Cada KPI deve possuir:

- * label;
- * valor principal;
- * ícone discreto;
- * informação auxiliar;
- * tooltip explicativo;
- * estado de carregamento;
- * tratamento para valor indisponível.

Exemplo visual:

```
```text
Taxa de churn
16,4%
82 de 500 contas
```
```

Quando houver comparação temporal válida, mostre variação:

```
```text
↓ 2,1 p.p. em relação ao período anterior
```

```

Não invente comparações caso a API não forneça esses dados.

Não use vermelho automaticamente em todo valor negativo. A cor deve refletir o contexto do indicador.

Cards e componentes

Padronize os cards com:

```
```css
background: #FFFFFF;
border: 1px solid #E2E8F0;
border-radius: 12px;
box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
```
```

No hover, use no máximo uma alteração discreta de borda ou sombra.

Todos os cards devem ter:

- * header consistente;
- * título;
- * subtítulo opcional;
- * área de ações;
- * conteúdo;
- * estado vazio;
- * estado de erro;
- * estado de carregamento.

Crie componentes CSS reutilizáveis em vez de repetir estilos específicos para cada bloco.

Gráficos

Reorganize os gráficos para facilitar a leitura executiva.

Primeira linha analítica

Dê maior destaque para:

- * evolução do churn;
- * evolução da receita perdida.

Utilize uma área maior, ocupando aproximadamente dois terços da largura, e ao lado apresente:

- * motivos de churn;
- * resumo de risco;

* ou distribuição por plano.

Segunda linha

Apresente:

- * churn por plano;
- * churn por indústria;
- * churn por país.

Terceira linha

Apresente:

- * uso do produto;
- * suporte;
- * erros;
- * satisfação.

Todos os gráficos devem possuir:

- * título claro;
- * subtítulo explicando a métrica;
- * menu de ações, quando necessário;
- * tooltip;
- * legenda legível;
- * cores consistentes;
- * altura adequada;
- * responsividade;
- * estado vazio;
- * estado de carregamento.

Não use gráficos de pizza quando houver muitas categorias.

Prefira:

- * barras horizontais;
- * linhas;
- * áreas;
- * barras agrupadas;
- * barras empilhadas;
- * dispersão;
- * heatmaps quando realmente úteis.

Limite as categorias exibidas e agrupe valores menores como "Outros" quando isso melhorar a leitura.

Configuração visual do Plotly

Centralize a configuração padrão dos gráficos JavaScript.

Use uma configuração semelhante a:

```
```javascript
const defaultPlotlyLayout = {
 paper_bgcolor: "transparent",
 plot_bgcolor: "transparent",
 font: {
 family: "Inter, system-ui, sans-serif",
 color: "#475569",
 size: 12
 },
 margin: {
 l: 48,
 r: 20,
 t: 28,
 b: 48
 },
 hoverlabel: {
 bgcolor: "#0F172A",
 bordercolor: "#0F172A",
 font: {
 color: "FFFFFF"
 }
 }
};
```
```

Remova elementos visuais desnecessários da toolbar do Plotly.

Mantenha disponível apenas o que fizer sentido, como exportação de imagem e reset de zoom.

Utilize:

```
```javascript
responsive: true
```
```

Não repita configurações completas em cada gráfico. Crie helpers reutilizáveis.

Seção "O que exige atenção"

Inclua na página inicial uma seção executiva intitulada:

```
```text
O que exige atenção
```
```

```

Ela deve apresentar, com base nos dados já disponíveis:

- \* contas críticas;
- \* maior motivo de churn;
- \* segmento com maior taxa de churn;
- \* plano com maior MRR perdido;
- \* aumento de tickets, quando mensurável;
- \* contas de alto valor com risco elevado.

Não invente insights.

Caso a API ainda não forneça todos esses dados, utilize apenas os dados disponíveis ou adapte a apresentação sem modificar indevidamente a lógica do backend.

Apresente os insights como cards compactos, por exemplo:

```
```text
Risco crítico
12 contas precisam de atenção imediata
Ver contas
```
```

## Contas em risco

A seção de contas em risco deve ser uma das áreas mais profissionais da solução.

Inclua:

- \* título;
- \* descrição;
- \* resumo por classificação;
- \* gráfico ou distribuição por risco;
- \* filtros rápidos;
- \* tabela;
- \* exportação.

Crie filtros rápidos:

```
```text
Todas
Críticas
Alto risco
Alto valor
Sem uso recente
Baixa satisfação
```
```

Preserve o filtro original enviado para a API ou faça o filtro local quando os dados já estiverem carregados.

### Tabela de risco

Melhore a tabela para apresentar inicialmente apenas as informações mais importantes:

- \* conta;
- \* plano;
- \* MRR;
- \* uso recente;
- \* tickets;
- \* satisfação;
- \* risk score;
- \* priority score;
- \* sinais;
- \* ação.

Informações menos prioritárias podem aparecer:

- \* em colunas opcionais;
- \* no detalhamento;
- \* em tooltip;
- \* em painel lateral;
- \* em linha expandida.

Use badges para classificação:

```
```text
Crítico
Alto
Médio
Baixo
```
```

Use cores suaves, nunca fundos totalmente saturados.

Exemplo:

```
```css
.badge-risk-critical {
  color: #991B1B;
  background: #FEE2E2;
}
```
```

Deixe o cabeçalho fixo quando a tabela possuir rolagem vertical.

Permita rolagem horizontal em telas menores.

Use estado de hover discreto nas linhas.

## Página de exploração de contas

Reestruture `/accounts` para funcionar como uma tela profissional de exploração.

Inclua:

- \* título;
- \* quantidade de resultados;
- \* busca;
- \* filtros;
- \* ordenação;
- \* botão para limpar filtros;
- \* tabela;
- \* paginação;
- \* estado vazio.

Cada conta deve ter:

- \* nome em destaque;
- \* ID em texto discreto;
- \* indústria;
- \* país;
- \* plano;
- \* status;
- \* MRR;
- \* risco;
- \* última atividade, quando disponível;
- \* ação "Ver detalhes".

Evite exibir todos os campos de uma vez.

## Detalhamento da conta

Reestruture `/accounts/<account\_id>` como uma visão de Customer Success.

### Cabeçalho

Apresente:

- \* nome da conta;
- \* account\_id;
- \* status;
- \* plano;

- \* indústria;
- \* país;
- \* data de cadastro;
- \* score de risco;
- \* score de prioridade;
- \* MRR;
- \* ARR.

Inclua uma ação para voltar à lista.

### ### Resumo

Crie cards para:

- \* receita;
- \* uso;
- \* suporte;
- \* churn;
- \* renovação.

### ### Navegação interna

Organize o conteúdo em abas ou seções:

```
```text
Resumo
Assinaturas
Uso do produto
Suporte
Churn e reativação
Linha do tempo
```
```

Pode usar navegação por abas via JavaScript simples.

Não faça novas requisições desnecessárias caso os dados já estejam carregados.

### ### Linha do tempo

A linha do tempo deve ser visualmente organizada, contendo:

- \* data;
- \* tipo do evento;
- \* título;
- \* descrição;
- \* ícone;
- \* cor contextual discreta.

Exemplos de eventos:

- \* cadastro;
- \* assinatura iniciada;
- \* upgrade;
- \* downgrade;
- \* ticket;
- \* churn;
- \* reativação;
- \* encerramento.

## Estados de carregamento

Crie skeleton loaders para:

- \* KPIs;
- \* gráficos;
- \* tabelas;
- \* informações de conta.

Não apresente apenas “Carregando...” em texto.

O skeleton deve ser discreto e desaparecer quando os dados forem carregados.

## Estados vazios

Quando uma consulta não retornar dados, apresente um estado vazio profissional:

```
```text
Nenhum dado encontrado

Não existem registros para os filtros selecionados.
Tente remover ou ajustar os filtros.
```
```

Inclua botão “Limpar filtros” quando aplicável.

## Tratamento de erros

Crie mensagens claras para falhas de API:

```
```text
Não foi possível carregar os dados

Ocorreu um problema ao consultar as informações.
Tente atualizar a página.
```
```

Inclua botão "Tentar novamente".

Não mostre stack trace, SQL ou caminho interno do computador na interface.

## ## Toasts e feedbacks

Implemente um sistema simples de toast para:

- \* filtros aplicados;
- \* filtros removidos;
- \* exportação iniciada;
- \* dados atualizados;
- \* erro na API;
- \* falha ao carregar gráfico.

Utilize JavaScript próprio ou recurso já existente no projeto.

Os toasts devem desaparecer automaticamente e permitir fechamento manual.

## ## Responsividade

Garanta boa experiência em:

- \* desktop grande;
- \* notebook;
- \* tablet;
- \* celular.

### ### Desktop

- \* sidebar fixa;
- \* múltiplas colunas;
- \* gráficos lado a lado;
- \* filtros em linha ou grid.

### ### Tablet

- \* sidebar recolhível;
- \* cards em duas colunas;
- \* gráficos em uma ou duas colunas;
- \* tabela com rolagem.

### ### Celular

- \* menu lateral em drawer;
- \* topbar simplificada;
- \* cards em uma coluna;
- \* gráficos em uma coluna;

- \* filtros em painel;
- \* botões com área de toque adequada;
- \* tabela com rolagem horizontal.

Utilize breakpoints consistentes.

## ## Acessibilidade

Implemente:

- \* contraste adequado;
- \* foco visível em links, botões e inputs;
- \* labels associadas aos campos;
- \* `aria-label` quando necessário;
- \* navegação por teclado;
- \* botões com texto ou descrição acessível;
- \* ícones decorativos ignorados por leitores de tela;
- \* estrutura semântica de headings;
- \* mensagens de erro compreensíveis.

Não remova o outline sem criar um estado de foco substituto.

## ## Organização do CSS

Evite manter um único arquivo CSS desorganizado.

Pode manter `styles.css`, mas estruture-o claramente:

```
```text
1. Variáveis
2. Reset e base
3. Layout
4. Sidebar
5. Topbar
6. Componentes
7. Formulários
8. KPIs
9. Gráficos
10. Tabelas
11. Badges
12. Estados
13. Utilitários
14. Responsividade
```
```

Utilize variáveis CSS:

```
```css
```



```
:root {
  --color-primary-900: #0F172A;
  --color-primary-800: #172033;
  --color-brand-600: #2563EB;
  --color-brand-700: #1D4ED8;
  --color-background: #F5F7FB;
  --color-surface: #FFFFFF;
  --color-border: #E2E8F0;
  --color-text: #0F172A;
  --color-text-secondary: #475569;
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 12px;
  --shadow-card: 0 1px 2px rgba(15, 23, 42, 0.04);
}
...
```

Evite estilos inline nos templates, salvo quando valores precisarem ser definidos dinamicamente.

Organização dos templates

Utilize herança Jinja2.

O `base.html` deve concentrar:

- * estrutura HTML;
- * sidebar;
- * topbar;
- * importações comuns;
- * área principal;
- * toast container;
- * scripts globais;
- * blocos Jinja.

Crie partials ou componentes reutilizáveis quando ajudar:

```
```text
templates/components/
■■■ sidebar.html
■■■ topbar.html
■■■ filters.html
■■■ kpi_card.html
■■■ empty_state.html
■■■ error_state.html
■■■ loading_state.html
```
```

Não crie fragmentação excessiva, mas evite duplicação.

JavaScript

Organize o JavaScript por responsabilidade.

Crie helpers para:

- * chamadas à API;
- * tratamento de erros;
- * loading;
- * formatação monetária;
- * formatação percentual;
- * formatação de datas;
- * configuração Plotly;
- * filtros;
- * toasts;
- * tabelas;
- * debounce da busca;
- * navegação mobile.

Não duplique lógica de `fetch` em todos os arquivos.

Sugestão:

```
```text
static/js/
■■■ api.js
■■■ ui.js
■■■ formatters.js
■■■ charts.js
■■■ dashboard.js
■■■ accounts.js
■■■ account-detail.js
```
```

Adapte a estrutura sem quebrar os scripts existentes.

Formatação

Padronize:

Moeda

```
```text
$ 1,234.56
```
```

Percentual

```
```text
16.4%
```
```

Número

```
```text
1,250
```
```

Datas

Para visão resumida:

```
```text
Jul 19, 2026
```
```

Para períodos:

```
```text
Jan 2026 – Jul 2026
```
```

Caso o projeto já possua um padrão definido, mantenha consistência.

Microinterações

Utilize apenas microinterações discretas:

- * mudança de fundo em hover;
- * transição de borda;
- * abertura suave da sidebar;
- * expansão de filtros;
- * aparecimento de toast;
- * skeleton loader.

Utilize transições entre 150 e 250 ms.

Respeite:

```
```css
@media (prefers-reduced-motion: reduce)
```
```

Página de saúde ou status

Caso já exista uma visualização de saúde da API, estilize-a de forma simples.

Não precisa criar uma página técnica nova caso não exista.

Não fazer

Não faça as seguintes alterações:

- * não altere a estrutura do banco;
- * não exclua tabelas;
- * não modifique registros;
- * não altere regras de churn;
- * não altere pesos do score sem necessidade;
- * não altere endpoints que já são consumidos;
- * não troque Flask por outro framework;
- * não introduza TypeScript;
- * não introduza build complexo;
- * não adicione autenticação;
- * não transforme tudo em uma SPA;
- * não use dados fictícios;
- * não deixe componentes sem conexão com a API;
- * não crie apenas um mockup visual;
- * não remova recursos existentes.

Melhorias permitidas no backend

Faça alterações no backend somente quando forem necessárias para sustentar a interface, por exemplo:

- * fornecer metadados úteis;
- * melhorar mensagens de erro;
- * fornecer paginação;
- * retornar total de registros;
- * retornar filtros disponíveis;
- * reduzir requisições duplicadas;
- * melhorar performance;
- * criar um endpoint agregado para insights.

Toda alteração deve preservar a compatibilidade com o funcionamento atual.

Processo de implementação

Execute a tarefa nesta sequência:

Etapa 1 – Auditoria

- * liste páginas, templates, CSS e JavaScript existentes;
- * identifique problemas de layout;

- * identifique duplicações;
- * identifique comportamentos que não podem ser quebrados;
- * registre brevemente as decisões.

Etapa 2 – Design system

- * crie variáveis CSS;
- * defina tipografia;
- * defina espaçamentos;
- * defina cards;
- * defina botões;
- * defina inputs;
- * defina badges;
- * defina estados;
- * defina grid;
- * defina breakpoints.

Etapa 3 – Shell principal

- * reorganize `base.html`;
- * crie sidebar;
- * crie topbar;
- * implemente menu responsivo;
- * implemente área principal.

Etapa 4 – Dashboard

- * reformule filtros;
- * reformule KPIs;
- * reorganize gráficos;
- * melhore tabela de risco;
- * implemente estados de loading, vazio e erro.

Etapa 5 – Contas

- * reformule listagem;
- * reformule detalhe;
- * melhore linha do tempo;
- * melhore navegação interna.

Etapa 6 – Validação

- * execute a aplicação;
- * acesse todas as páginas;
- * teste filtros;
- * teste gráficos;
- * teste busca;
- * teste tabela;

- * teste exportação;
- * teste detalhes;
- * teste responsividade;
- * verifique erros no console;
- * verifique erros no terminal.

Critérios de aceite

A tarefa será considerada concluída somente quando:

1. o dashboard possuir aparência profissional;
2. a sidebar estiver funcional;
3. a topbar estiver consistente;
4. a hierarquia visual estiver clara;
5. os KPIs estiverem organizados;
6. os gráficos estiverem legíveis;
7. os filtros estiverem claros;
8. a tabela de risco estiver utilizável;
9. as páginas de contas estiverem reformuladas;
10. os dados reais continuarem sendo exibidos;
11. os filtros continuarem funcionando;
12. os gráficos continuarem sendo carregados;
13. as exportações continuarem funcionando;
14. não houver erros de JavaScript no console;
15. não houver erros de Flask no terminal;
16. a aplicação funcionar em desktop, tablet e celular;
17. os estados de loading, vazio e erro estiverem implementados;
18. os estilos estiverem organizados;
19. não houver duplicação excessiva de código;
20. o README estiver atualizado com a nova estrutura visual.

Entrega final

Não apenas explique o que deveria ser feito.

Edite efetivamente os arquivos do projeto.

Ao concluir, apresente:

1. resumo da auditoria realizada;
2. arquivos criados;
3. arquivos alterados;
4. componentes visuais implementados;
5. melhorias de responsividade;
6. melhorias de acessibilidade;
7. funcionalidades preservadas;
8. testes realizados;
9. eventuais limitações;

10. instruções para executar a aplicação.

O resultado final deve parecer um produto SaaS analítico pronto para ser apresentado em uma demonstração executiva, mesmo continuando sendo um protótipo local.